基因组信息学
实验总结报告

年级：2023

专业：生物信息学

姓名：龚志远

学号：2330416033

目录

摘要 4
1. 材料和方法 5
1.1. 分析平台 5
1.1.1. 硬件平台： 5
1.1.2. 操作系统： 5
1.1.3. 分析软件： 5
1.1.4. 数据库资源： 6
1.2. 研究对象 6
1.3. 方法 6
1.3.1. 基因组测序模拟 6
1.3.2. 序列组装 10
1.3.3. 基因组注释之同源搜索 12
1.3.4. 从头预测与基因结构建模 16
1.3.5. 基因组注释之 DNA 元件预测 17
1.3.6. 基因组数据可视化 25
2. 结果 28
2.1. 基因组测序模拟 28
2.1.1. 结果解读 29
2.2. 序列组装 29
2.2.1. FastQC 质量控制 29
2.2.2. QUAST 结果（综合 3x10x33x） 29
2.3. 基因组注释之同源搜索 33
2.3.1. Samtools 结果 33
2.3.2. 同源搜索结果的评价与比较 33
2.4. 从头预测与基因结构建模 35
2.4.1. 基因预测概况 35
2.4.2. 序列提取结果 36
2.4.3. 同源性鉴别结果 36
2.4.4. 准确性评估统计 36
2.5. 基因组注释之 DNA 元件预测 38
2.5.1. 预测元件到最近基因的距离分布 38
2.5.2. PWM 得分与距离的关联散点图 39
2.5.3. PWM 得分频数分布直方图 40
2.6. 基因组数据可视化 40
2.6.1. 基因结构预测的准确性验证 41
2.6.2. 可视化工具在纠错中的应用 42

3. 总结与讨论 42
3.1. 基因组测序模拟 42
3.1.1. 覆盖度对基因组覆盖率的影响 42
3.1.2. 模拟参数的多因素方差分析 42
3.2. 序列组装 42
3.2.1. 测序深度对组装连续性的决定性作用 42
3.2.2. Contig 与 Scaffold 的策略权衡 43
3.2.3. GC 偏倚对组装的影响 43
3.3. 基因组注释之同源搜索 43
3.3.1. 两种 BLAST 结果转换策略的差异分析 43
3.3.2. 测序模拟数据的比对特征 43
3.4. 从头预测与基因结构建模 44
3.4.1. Ab Initio 预测的准确性瓶颈 44
3.4.2. BLASTP 验证的必要性 44
3.5. 基因组注释之 DNA 元件预测 44
3.5.1. 预测结果的生物学意义分析 44
3.5.2. 模型适用性与跨物种保守性探讨 45
3.5.3. 阈值设定策略的讨论 45
3.6. 基因组数据可视化 45
3.6.1. 大数据可视化的核心机制：Range Request 技术的必要性 45
3.6.2. 可视化在多组学整合中的方法论意义 45

摘要

本次实验选取酿酒酵母（*Saccharomyces cerevisiae*，菌株 HLJ1 Assembly v2）作为研究对象，旨在通过全流程的生物信息学分析掌握基因组学的核心技术 。首先使用 ART 软件对基因组进行测序模拟，结果显示覆盖度（Coverage）是影响基因组理论覆盖率的决定性因素：当测序深度从 1x 增加到 11x 时，理论覆盖率从 63.21% 跃升至 99.99%，验证了深度换广度的策略 。

随后进行序列组装与评估。利用 FastQC 对模拟数据进行质量控制，确认数据质量优良（Q30 以上）且无接头污染 。使用 SOAPdenovo2 进行组装，并由 QUAST 进行评估 。结果表明，33x 测序深度的 Scaffold NGA50 达到约 540kb，显著优于 Contig 的连续性，证明了长插入片段在构建基因组框架中的重要作用 。

在基因组注释阶段，采用了三种策略。同源搜索方面，利用 BLAST+ 将 UniProt 蛋白序列比对至基因组，对比发现 Perl 脚本转换策略在还原基因结构（Transcript level Sensitivity 83.0%）上优于 Python 脚本 。从头预测方面，使用 Augustus 软件预测出 5413 个基因，其碱基水平灵敏度高达 97.4%，但内含子预测准确度较低，需结合同源证据修正 。DNA 元件预测方面，构建 PWM 模型扫描全基因组，发现 TATA-box 显著富集在转录起始位点（TSS）上游 0-200bp 区域，验证了该元件的生物学保守性 。

最后，搭建 JBrowse 本地可视化平台，成功整合了参考序列、从头预测、同源搜索及 DNA 元件等多维数据轨道，直观验证了多源证据对基因结构注释的支撑作用

1. 材料和方法

1.1. 分析平台

1.1.1. 硬件平台：

1.1.1.1. 服务器端：

远程 Linux 计算服务器
处理器 (CPU)：Intel(R) Core(TM)2 Duo CPU T7700 @ 2.40GHz
核心数：16 逻辑核心 (1 Socket × 16 Cores/Socket × 1 Thread/Core)
主频：2.40 GHz
内存 (RAM)：16 GB

1.1.1.2. 客户端：

本地 Mac 个人电脑
处理器：Apple M4
核心数：10 物理核心，10 逻辑核心
内存：16GB
存储：512GB

1.1.2. 操作系统：

1.1.2.1. 服务器 OS

Ubuntu 20.04.6 LTS (Focal Fossa)
内核版本：Linux 5.15.0-139-generic

1.1.2.2. 本地 OS：

MacOS Sequoia 15.6

1.1.3. 分析软件：

ART (ART_Illumina)：v2.5.8 (June 7, 2016) —— 用于基因组测序模拟
FastQC：v0.11.9 —— 用于测序数据质量控制
SOAPdenovo2：v1.05 (released on July 29th, 2010) —— 用于短序列组装
QUAST：v5.2.0 (3d87c606) —— 用于组装结果评估
Bowtie2：v2.3.5.1 —— 用于构建索引及序列比对
Samtools：v1.10 —— 用于 BAM/SAM 文件处理及统计
Bedtools：v2.27.1 —— 用于基因组区间操作及序列提取
BLAST+ (blastn)：v2.15.0+ —— 用于同源搜索
Augustus：v3.3 —— 用于真核生物基因从头预测
GffCompare：v0.11.2 —— 用于基因结构比较与评估
Python：v3.11.4 —— 用于编写数据处理脚本
R Language：v3.6.3 (2020-02-29, "Holding the Windsock") —— 用于统计分析与绘图

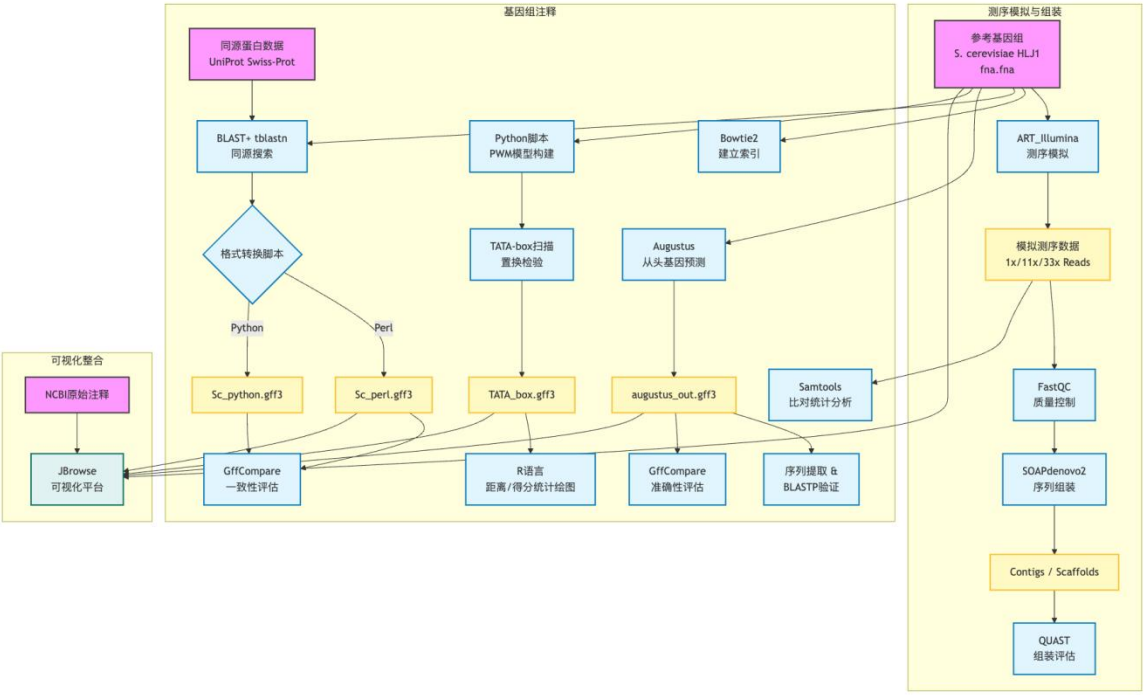
1.1.4. 数据库资源：

Genbank 数据库 Genome 子库: https://www.ncbi.nlm.nih.gov/genome
Genbank 数据库 SRA 子库: https://www.ncbi.nlm.nih.gov/sra
illumina 数据库:
https://support.illumina.com/sequencing/sequencing_software/igenome.html
UniProt 数据库: https://www.uniprot.org/
EPD 数据库: https://epd.epfl.ch/promoter_elements.php

1.2. 研究对象

物种名称：酿酒酵母 (Saccharomyces cerevisiae)
菌株/组装版本：Strain S288C / HLJ1 Assembly v2 (GCA_903819155.2)
数据来源：NCBI Genome / GenBank

1.3. 方法



1.3.1. 基因组测序模拟

1.3.1.1. 下载基因及注释文件

访问 genome 数据库（https://www.ncbi.nlm.nih.gov/datasets/genome/），搜索酿酒酵母（Saccharomyces cerevisiae S288C），本实验选择 HLJ1 Assembly v2

Genome assembly HLJ1 Assembly v2

Download datasets API FTP

Submitted GenBank assembly	GCA_903819155.2
Taxon	Saccharomyces cerevisiae (brewer's yeast)
Strain	HLJ1
Assembly type	haploid
Submitter	BOISE STATE UNIVERSITY
Date	Nov 16, 2020

下载时勾选 Genome sequences (FASTA)，Sequence and annotation (GBFF)，解压后重命名并保存,fna 文件命名为 fna.fna,gbff 文件重命名为 gff.gff

1.3.1.2. 测序实验

使用 win+R 快捷键输入 powershell 进入命令行，输入以下代码

```
sftp user@10.8.12.35#回车后输入密码
mkdir Geneinfo
cd Geneinfo
lcd Geneinfo
put fna.fna
put gff.gff
```

打开新页面，输入以下代码

```
ssh user@10.8.12.35#回车后输入密码
cd Geneinfo
art_illumina -ss HS25 -sam -i fna.fna -p -l 100 -f 1 -m 200 -s 10 -o
sim_l100_f1_m200
```

运行结果如下

```
(base) user@0071:~/Geneinfo$ art_illumina -ss HS25 -sam -i fna -p -l 100 -f 1 -m 200 -s 10 -o sim_l100_f1_m200

    ====================ART====================
             ART_Illumina (2008-2016)
           Q Version 2.5.8 (June 7, 2016)
     Contact: Weichun Huang <whduke@gmail.com>
    ----------------------------------------

                   Paired-end sequencing simulation

Total CPU time used: 2.88

The random seed for the run: 1758250701

Parameters used during run
        Read Length:    100
        Genome masking 'N' cutoff frequency:    1 in 100
        Fold Coverage:            1X
        Mean Fragment Length:     200
        Standard Deviation:       10
        Profile Type:             Combined
        ID Tag:

Quality Profile(s)
        First Read:   HiSeq 2500 Length 126 R1 (built-in profile)
        First Read:   HiSeq 2500 Length 126 R2 (built-in profile)

Output files
```

head -n 20 sim_l100_f1_m200.sam

结果如下图

```
(base) user@0071:~/Geneinfo$ head -n 20 sim_l100_f1_m200.sam
@HD     VN:1.4  SO:unsorted
@SQ     SN:LR813500.2 Saccharomyces cerevisiae strain HLJ1 genome assembly, chromosome: 1       LN:229169
@SQ     SN:LR813501.2 Saccharomyces cerevisiae strain HLJ1 genome assembly, chromosome: 2       LN:797722
@SQ     SN:LR813502.2 Saccharomyces cerevisiae strain HLJ1 genome assembly, chromosome: 3       LN:290009
@SQ     SN:LR813503.2 Saccharomyces cerevisiae strain HLJ1 genome assembly, chromosome: 4       LN:1458639
@SQ     SN:LR813504.2 Saccharomyces cerevisiae strain HLJ1 genome assembly, chromosome: 5       LN:560417
@SQ     SN:LR813505.2 Saccharomyces cerevisiae strain HLJ1 genome assembly, chromosome: 6       LN:273789
@SQ     SN:LR813506.2 Saccharomyces cerevisiae strain HLJ1 genome assembly, chromosome: 7       LN:1066793
@SQ     SN:LR813507.2 Saccharomyces cerevisiae strain HLJ1 genome assembly, chromosome: 8       LN:531779
@SQ     SN:LR813508.2 Saccharomyces cerevisiae strain HLJ1 genome assembly, chromosome: 9       LN:433662
@SQ     SN:LR813509.2 Saccharomyces cerevisiae strain HLJ1 genome assembly, chromosome: 10      LN:709184
@SQ     SN:LR813510.2 Saccharomyces cerevisiae strain HLJ1 genome assembly, chromosome: 11      LN:660306
@SQ     SN:LR813511.2 Saccharomyces cerevisiae strain HLJ1 genome assembly, chromosome: 12      LN:1031746
@SQ     SN:LR813512.2 Saccharomyces cerevisiae strain HLJ1 genome assembly, chromosome: 13      LN:895159
@SQ     SN:LR813513.2 Saccharomyces cerevisiae strain HLJ1 genome assembly, chromosome: 14      LN:774029
@SQ     SN:LR813514.2 Saccharomyces cerevisiae strain HLJ1 genome assembly, chromosome: 15      LN:1070054
@SQ     SN:LR813515.2 Saccharomyces cerevisiae strain HLJ1 genome assembly, chromosome: 16      LN:919881
@SQ     SN:LR813516.2 Saccharomyces cerevisiae strain HLJ1 genome assembly, organelle: mitochondrion    LN:78472
@PG     ID:01   PN:ART_Illumina CL:art_illumina -ss HS25 -sam -i fna -p -l 100 -f 1 -m 200 -s 10 -o sim_l100_f1_m200 -rs
 1758250701
LR813500.2-2291 83      LR813500.2      216133  99      100=    =       216032  -201    TTCTAGAGGAACAAATTGATATTTTCAGATCA
GAATTCATCAAATATATGCACTTGGTAAAAACCAAGGATGTGATGAAGTTAAACTATTGTCTTTATGG    GGGGGGGGGEBGGG@GGEFG0DGGFGEFCG10GGGGGGGGGGGEGGGG
E>GGGGGGGGFCGGGGGGGGG1GGGGGGGGGGFGGGGDGG0GGGBG1BBCCB
```

将表格保存为 data.txt

1.3.1.3. R 语言作图

打开 Rstudio，输入以下代码

```
rm(list=ls())
library(ggplot2)
# 定义数据文件
file <- "data.txt"
# 读取数据文件
data <- read.table(file, header = TRUE, sep = "\t")
# 查看数据文件内容以及行列数
data
nrow(data)
ncol(data)
```

```
# 多因素分析，且考虑交互作用
re <- aov(c ~ l * f * m, data = data)
# 查看和保存统计分析结果
summary(re)
```

结果如下图

```
> summary(re)
            Df  Sum Sq Mean Sq F value Pr(>F)
l            1 0.00000 0.00000   0.000  1.000
f            1 0.14998 0.14998   6.188  0.131
m            1 0.00000 0.00000   0.000  1.000
l:f          1 0.00000 0.00000   0.000  1.000
l:m          1 0.04732 0.04732   1.952  0.297
f:m          1 0.02000 0.02000   0.825  0.460
Residuals    2 0.04848 0.02424
```

```
# 获取并查看 x 坐标轴最大值
x_max <- max(data["f"])
x_max
# 开始绘图
p <- ggplot(data = data, aes(x = f, y = c)) +
  scale_x_continuous(breaks = c(0:x_max)) +
  geom_point(
    aes(size = l, fill = m),   # 将颜色映射改为填充映射
    shape = 21,                # 空心圆形状
    color = "black",           # 统一边框颜色
    alpha = 0.7                # 透明度调整
  ) +
  scale_size_continuous(name = "l", range = c(5, 10)) +
  scale_fill_gradient(         # 填充渐变替代颜色渐变
    name = "m",
    low = "#DCE2F1",
    high = "#FDE6F0"
  ) +
  geom_line(aes(x = f, y = c), color = "black") +
  ggtitle("c-c relevance curve\nplot by gzy,2025-09-16") +
  labs(
    x = "fold",
    y = "coverage",
    legend.fill = c("m", "l")
  ) +
  theme(
```

```
    title = element_text(size = 14, face = "bold"),
    axis.title.x = element_text(size = 14, face = "bold"),
    axis.title.y = element_text(size = 14, face = "bold"),
    axis.text.x = element_text(size = 12, face = "bold"),
    axis.text.y = element_text(size = 12, face = "bold"),
    legend.title = element_text(size = 12, face = "bold"),
    legend.text = element_text(size = 12, face = "bold"),
    plot.margin = unit(c(1, 1, 1, 1), "lines"),
    plot.title = element_text(hjust = 0.5)
  )
# 输出 PDF 格式绘图文件
ggsave("ggplot2_bubble_ngs_simulation_by_gzy.pdf", p, width = 8, height = 8)
```

1.3.2. 序列组装

进入目录查看 pdf 和 html 分别进行短插入片段库和长插入片段库测序模拟

```
cd Geneinfo
```

测序模拟，覆盖度使用 3，10，33。前缀为覆盖度大小和长短插入片段

```
art_illumina -ss HS25 -sam -i fna.fna -p -l 125 -f 3 -m 200 -s 10 -o 3_short
art_illumina -ss HS25 -sam -i fna.fna -p -l 125 -f 3 -m 2500 -s 50 -o 3_long
art_illumina -ss HS25 -sam -i fna.fna -p -l 125 -f 10 -m 200 -s 10 -o 10_short
art_illumina -ss HS25 -sam -i fna.fna -p -l 125 -f 10 -m 2500 -s 50 -o 10_long
art_illumina -ss HS25 -sam -i fna.fna -p -l 125 -f 33 -m 200 -s 10 -o 33_short
art_illumina -ss HS25 -sam -i fna.fna -p -l 125 -f 33 -m 2500 -s 50 -o 33_long
```

进行质量控制

```
mkdir fastqc_out
fastqc -o fastqc_out 3_long1.fq 3_short2.fq 3_long1.fq 3_long2.fq
fastqc -o fastqc_out 10_long1.fq 10_short2.fq 10_long1.fq 10_long2.fq
fastqc -o fastqc_out 33_long1.fq 33_short2.fq 33_long1.fq 33_long2.fq
```

编写 cfg 文件

```
vim lib.cfg
```

内容如下

```
max_rd_len=125
[LIB]
avg_ins=200
reverse_seq=0
asm_flags=3
rd_len_cutoff=125
rank=1
pair_num_cutoff=3
map_len=32
q1=short1.fq
q2=short2.fq
[LIB]
```

```
avg_ins=2500
reverse_seq=1
asm_flags=2
rank=2
pair_num_cutoff=5
map_len=35
rd_len_cutoff=125
q1=long1.fq
q2=long2.fq
```

安装 soapdenovo 并运行

```
sudo apt install soapdenovo
soapdenovo-63mer all -s lib.cfg -K 31 -o soap_out
```

编写批量脚本(要避免爆内存)

```
vim run_soap.sh
KMER=13
THREADS=2
PREFIX_LIST=("3_" "10_" "33_")
OUTDIR="soap_out"
CFG_TEMPLATE="lib.cfg"

mkdir -p ${OUTDIR}

for prefix in "${PREFIX_LIST[@]}"; do
	cfg="${prefix}lib.cfg"
	cp ${CFG_TEMPLATE} ${cfg}
	sed -i "s#short1.fq#${prefix}short1.fq#g" ${cfg}
	sed -i "s#short2.fq#${prefix}short2.fq#g" ${cfg}
	sed -i "s#long1.fq#${prefix}long1.fq#g"    ${cfg}
sed -i "s#long2.fq#${prefix}long2.fq#g"    ${cfg}

	soapdenovo-63mer all \
		-s ${cfg} \
		-K ${KMER} \
		-p ${THREADS} \
		-o ${OUTDIR}/${prefix}soap_out
done
```

编写进行批量评估

```
vim run_quast_cmp.sh
```

以下为脚本内容

```
BASE=~/Geneinfo
REF_FA=$BASE/fna.fna
REF_GFF=$BASE/gff.gff
SOAP=$BASE/soap_out
```

```
OUT=$BASE/quast_cmp
THREADS=8

mkdir -p "$OUT"

C3="$SOAP/3_soap_out.contig"
C10="$SOAP/10_soap_out.contig"
C33="$SOAP/33_soap_out.contig"
S3="$SOAP/3_soap_out.scafSeq"
S10="$SOAP/10_soap_out.scafSeq"
S33="$SOAP/33_soap_out.scafSeq"

quast.py -o "$OUT" -t "$THREADS" -r "$REF_FA" -g "$REF_GFF" \
  -l "3x_contig,10x_contig,33x_contig,3x_scaffold,10x_scaffold,33x_scaffold" \
  "$C3" "$C10" "$C33" "$S3" "$S10" "$S33"

echo "[OK] QUAST done -> $OUT"
```

1.3.3. 基因组注释之同源搜索

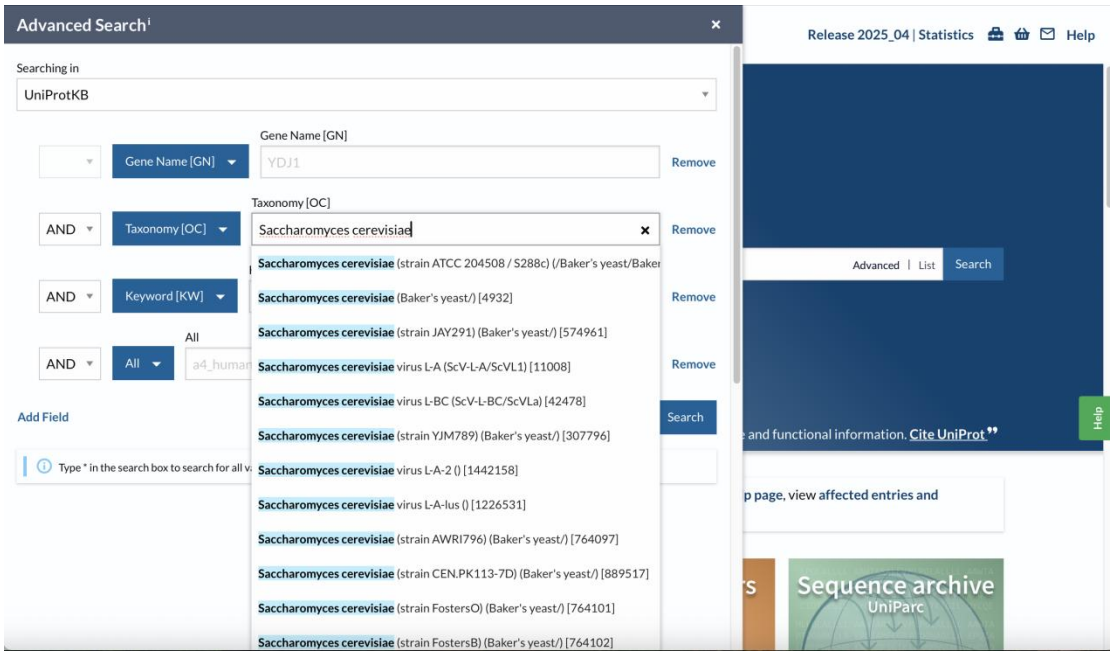
1.3.3.1. 数据准备与比对

为了进行全基因组的同源基因搜索，使用了 Saccharomyces cerevisiae 的参考基因组（文件 fna.fna）和下载的已审核蛋白质序列（Swiss-Prot）数据。以下是实验步骤：

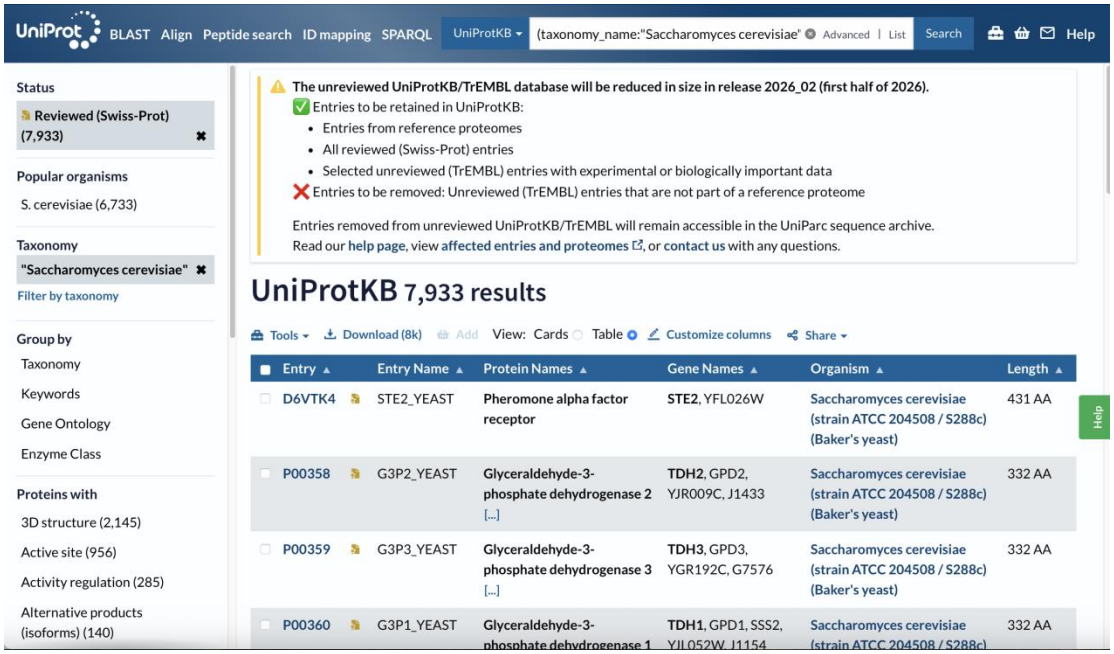
1.3.3.1.1.数据准备

下载了 Saccharomyces cerevisiae 近缘物种的已审核蛋白质序列，并将其保存为 proteins_uniprot/scere_reviewed.fa.gz。这一步骤确保了可以使用这些蛋白质序列进行比对。

进入 uniprotkb，打开 advance 搜索界面，搜索 Saccharomyces cerevisiae



将 reviewed 勾选，下载 7933 个蛋白质



将下载的数据通过 sftp 传输到服务器。存储到 proteins_uniprot 目录

1.3.3.1.2.创建基因组索引

使用 bowtie2 创建基因组索引以供比对使用。命令如下：

```
# 创建基因组索引
bowtie2-build fna.fna Sc_index
```

1.3.3.1.3. 进行序列比对

选择了 samples/33_long1.fq 和 samples/33_long2.fq 这对文件，利用 bowtie2 对其进行比对，生成比对结果文件 paired.sam。在这一步骤中，使用了配对比对模式和最大插入片段长度参数（-I 0 -X 2000），以确保比对的准确性。

```
# 配对比对
bowtie2 -x Sc_index -1 samples/33_long1.fq -2 samples/33_long2.fq \
  -S paired_fix.sam -p 4 --rf -I 0 -X 2000
1554965 reads; of these:
  1554965 (100.00%) were paired; of these:
    1550392 (99.71%) aligned concordantly 0 times
    2112 (0.14%) aligned concordantly exactly 1 time
    2461 (0.16%) aligned concordantly >1 times
    ----
    1550392 pairs aligned concordantly 0 times; of these:
      1449407 (93.49%) aligned discordantly 1 time
    ----
    100985 pairs aligned 0 times concordantly or discordantly; of these:
      201970 mates make up the pairs; of these:
        1 (0.00%) aligned 0 times
        65289 (32.33%) aligned exactly 1 time
        136680 (67.67%) aligned >1 times
100.00% overall alignment rate
```

比对的结果表明，99.71%的配对比对未能一次性正确配对（concordantly 0 times），而约 93.49% 配对能够通过“discordantly 1 time”来对齐。整体比对率为 100%，这意味着大多数序列能够定位到基因组。这是因为选用的模拟数据，后续的流程继续进行。

1.3.3.2. 基因组比对结果处理

1.3.3.2.1.转换 SAM 格式为 BAM 格式

为了进行更高效的分析， 将 paired.sam 文件转换为 paired.bam 格式，并进行排序和索引处理，确保数据的便捷存取。

```
# 将 SAM 格式转换为 BAM 格式
samtools view -bS paired.sam -o paired.bam
# 对 BAM 文件进行排序
samtools sort paired.bam -o paired.sorted.bam
# 创建索引文件
samtools index paired.sorted.bam
```

1.3.3.2.2.使用 Samtools 进行统计分析

使用 samtools 进行各种统计分析，以了解比对的质量。这些分析包括：对 BAM 文件的基本统计信息、flagstat 结果、以及索引统计。

```
# 生成比对统计信息
```

```
samtools stats paired.sorted.bam > samtools.stats.out
# 生成 flagstat 结果
samtools flagstat paired.sorted.bam > samtools.flagstat.out
# 生成索引统计信息
samtools idxstats paired.sorted.bam > samtools.idxstats.out
#画图对结果进行可视化
sudo apt update
sudo apt install gnuplot
plot-bamstats -p ./plot-bamstats_out ./samtools.stats.out
```

1.3.3.3. 同源基因比对与分析

1.3.3.3.1.创建 BLAST 数据库

为了进行同源基因搜索，使用 makeblastdb 创建了本地的 BLAST 数据库。该数据库是基于 Saccharomyces cerevisiae 参考基因组的序列 fna.fna 创建的。

```
# 创建 BLAST 数据库
makeblastdb -in ~/Geneinfo/fna.fna -input_type fasta -dbtype nucl -title
Sc_genome -out ~/Geneinfo/Sc_genome_db
```

1.3.3.3.2.运行 BLAST 同源搜索

接下来，使用 tblastn 进行同源基因比对。运行了两次 tblastn，分别限制最大匹配序列为 1 和不做限制，并将结果输出到不同的文件中。

```
# 限定最大匹配序列为 1
tblastn -query proteins_uniprot/scere_reviewed.fa.gz -db
~/Geneinfo/Sc_genome_db \
  -evalue 1e-6 -outfmt 6 -max_target_seqs 1 -out
~/Geneinfo/tblastn_results.outfmt6
# 不限定最大匹配序列
tblastn -query proteins_uniprot/scere_reviewed.fa.gz -db
~/Geneinfo/Sc_genome_db \
  -evalue 1e-6 -outfmt 6 -out ~/Geneinfo/tblastn_results_all.outfmt6
```

1.3.3.3.3.转换 BLAST 结果为 GFF3 格式

使用 blast92gff3.pl 和 blast2gff.py 两个工具，将 tblastn 的结果转换为 GFF3 格式，并分别保存为 Sc_perl.gff3 和 Sc_python.gff3。

```
# 使用 Perl 脚本转换为 GFF3 格式
blast92gff3.pl tblastn_results.outfmt6 > Sc_perl.gff3
# 使用 Python 脚本转换为 GFF3 格式
python2 /usr/bin/blast2gff.py -b tblastn_results.outfmt6 > Sc_python.gff3
```

1.3.3.3.4.GFF3 结果修改与评估

为了符合实验要求，将 match 修改为 gene，将 HSP 修改为 exon，并保存为修改后的 GFF3 文件。随后，使用 gffcompare 评估了修改后的 GFF3 文件与原始 GFF 注释文件之间的匹配情况。

```
# 修改并保存为新的 GFF3 文件
sed 's/\tmatch\t/\tgene\t/g; s/\tHSP\t/\texon\t/g' Sc_perl.gff3 > Sc_perl_modified.gff3
sed 's/match_part/exon/g' Sc_python.gff3 > Sc_python_modified.gff3
# 使用 gffcompare 评估一致性
gffcompare -V -r gff.gff Sc_perl_modified.gff3 -o Sc_perl_eval
gffcompare -V -r gff.gff Sc_python_modified.gff3 -o Sc_python_eval
```

1.3.4. 从头预测与基因结构建模

1.3.4.1. 从头基因预测

使用 Augustus 软件，基于酿酒酵母（Saccharomyces cerevisiae S288C）的内建模型参数，对目标基因组序列进行全基因组预测。由于本地服务器计算资源受限，本步骤在高性能计算节点上完成。

```
nohup augustus --species=saccharomyces_cerevisiae_S288C --gff3=on fna.fna > augustus_out.gff3 2> run.log &
```

将预测结果传回主服务器

1.3.4.2. 预测结果的序列提取

为了验证预测结果的生物学意义，我们需要从 GFF3 文件中提取出预测基因的核苷酸序列和编码的蛋白质序列。

```
#建立基因组索引
samtools faidx fna.fna
#先筛选出 gene 特征行，避免 bedtools 提取多余特征
awk -F'\t' '$3=="gene" {print $0}' augustus_out.gff3 > augustus_genes_only.gff3
bedtools getfasta -fi fna.fna -bed augustus_genes_only.gff3 -fo predicted_genes.fa
#使用 awk 脚本从 GFF3 注释行中提取清洗后的蛋白序列
awk '/^ start gene/ {print ">"$4} /^ protein sequence =/ {flag=1; sub(/^.\[/,""); } flag {if(/\]/) is_end=1; else is_end=0; sub(/\]/,""); sub(/^ /,""); if(length($0)>0) print; if(is_end) flag=0;}' augustus_out.gff3 > predicted_proteins.fa
```

1.3.4.3. 同源性鉴别

利用 BLASTP 将预测的蛋白质序列与近缘物种的已知蛋白质数据库（UniProt）进行比对，以验证预测基因的真实性。

```
#建立 BLAST 蛋白质数据库 (输入文件来自实验三)
makeblastdb
-in ../03_Homology_Search/proteins_uniprot/uniprotkb_taxonomy_name_Saccharomyces_c_2025_10_23.fasta \
-input_type fasta -dbtype prot -title uniprot_prot_db -out uniprot_prot_db
#运行 BLASTP 比对
blastp -query predicted_proteins.fa -db uniprot_prot_db -out blastp.outfmt6 \
-evalue 1e-5 -outfmt 6 -max_target_seqs 1
```

1.3.4.4. 预测准确性评估

```
gffcompare -r ../gff.gff -o augustus_eval augustus_out.gff3
```

1.3.5. 基因组注释之 DNA 元件预测

1.3.5.1. 构建 PWM 模型与全基因组扫描

获取频率矩阵：选取教程中提供的植物启动子 TATA-box 的碱基频率矩阵。
构建 PWM 模型：编写 Python 程序，将原始的频率矩阵转换为位置权重矩阵（PWM）。其中背景概率设为 0.25。
滑动窗口扫描：利用构建好的 PWM 模型，以 1bp 为步长遍历酿酒酵母全基因组序列（fna.fna）。
显著性评估 (P-value)：为了降低假阳性率，对得分超过阈值 (理论最大值的 75%) 的序列片段进行置换检验。即将该片段随机打乱 100 次，计算随机序列得分超过原始得分的概率，仅保留 $P < 0.05$ 的高置信度结果。
创建 predict_tata 脚本

```
vim predict_tata.py
```

具体代码如下

```
import math
import random
import sys

# 1. 定义 TATA-box 的频率矩阵
# 为了防止 log(0)，添加一个极小的伪计数 0.001
freq_matrix = [
    {'A': 31.6, 'C': 24.5, 'G': 15.3, 'T': 28.6},
    {'A': 16.3, 'C': 60.2, 'G': 10.2, 'T': 13.3},
    {'A':  2.0, 'C':  3.0, 'G':  0.0, 'T': 94.9},
    {'A': 90.8, 'C':  2.1, 'G':  2.0, 'T':  5.1},
    {'A':  0.0, 'C':  0.0, 'G':  1.0, 'T': 99.0},
    {'A': 94.9, 'C':  0.0, 'G':  0.0, 'T':  5.1},
    {'A': 57.1, 'C':  0.0, 'G':  0.0, 'T': 42.9},
    {'A': 100.0,'C':  0.0, 'G':  0.0, 'T':  0.0},
    {'A': 27.6, 'C':  0.0, 'G':  2.0, 'T': 70.4},
    {'A': 69.4, 'C':  3.1, 'G': 13.3, 'T': 14.3},
    {'A': 11.2, 'C': 39.8, 'G': 37.8, 'T': 11.2},
    {'A': 24.5, 'C': 52.0, 'G': 21.4, 'T':  2.1}
]

# 背景概率 (假设基因组 GC 含量均等)
bg = 0.25
pseudo = 0.001

# 2. 构建 PWM - Log-Odds Score
pwm = []
for pos in freq_matrix:
```

```
    pwm_row = {}
    total = sum(pos.values())
    for base, percent in pos.items():
        # 计算概率 p
        p = (percent / 100.0)
        if p == 0: p = pseudo
        # 计算 Log-Odds: log2(p / background)
        score = math.log(p / bg, 2)
        pwm_row[base] = score
    pwm.append(pwm_row)

motif_len = len(pwm)

# 辅助函数：读取 FASTA
def read_fasta(file_path):
    genome = {}
    name = ""
    seq = []
    with open(file_path, 'r') as f:
        for line in f:
            line = line.strip()
            if line.startswith(">"):
                if name: genome[name] = "".join(seq).upper()
                name = line[1:].split()[0] # 获取序列 ID
                seq = []
            else:
                seq.append(line)
        if name: genome[name] = "".join(seq).upper()
    return genome

# 辅助函数：计算序列得分
def score_seq(sequence):
    s = 0
    if len(sequence) != motif_len: return -999
    for i in range(motif_len):
        base = sequence[i]
        if base not in ['A','C','G','T']: return -999 # 忽略 N
        s += pwm[i][base]
    return s

# 辅助函数：获取反向互补链
def rev_comp(seq):
    complement = {'A': 'T', 'C': 'G', 'G': 'C', 'T': 'A', 'N': 'N'}
```

```
    return "".join([complement.get(base, base) for base in seq[::-1]])

# 3. 主程序
genome_file = "../fna.fna"
output_file = "TATA_box_prediction.gff3"

print(f"Reading genome from {genome_file}...")
genome = read_fasta(genome_file)

# 设定一个初始硬阈值以减少计算量
# 理论最大得分
max_possible_score = sum([max(row.values()) for row in pwm])
threshold = max_possible_score * 0.75
print(f"Max score: {max_possible_score:.2f}, Pre-filter threshold: {threshold:.2f}")

results = []

print("Scanning genome (this may take a while)...")
with open(output_file, 'w') as out:
    out.write("##gff-version 3\n")

    for chrom, sequence in genome.items():
        seq_len = len(sequence)
        # 正链扫描
        for i in range(seq_len - motif_len + 1):
            segment = sequence[i : i+motif_len]
            if 'N' in segment: continue

            raw_score = score_seq(segment)

            # 只有得分够高才进行 P-value 计算
            if raw_score > threshold:
                # 简化的 P-value 计算：随机打乱当前片段 100 次
                # 如果打乱后的得分比原始得分高的次数越多，说明原始得分越
不显著
                shuffle_count = 100
                better_score_count = 0
                seg_list = list(segment)
                for _ in range(shuffle_count):
                    random.shuffle(seg_list)
                    shuffled_seq = "".join(seg_list)
                    if score_seq(shuffled_seq) >= raw_score:
                        better_score_count += 1
```

```
                    p_value = (better_score_count + 1) / (shuffle_count + 1)

                    # 筛选条件：P-value < 0.05
                    if p_value < 0.05:
                        # GFF3 格式: seqid source type start end score strand
phase attributes
                        attr = f"pvalue={p_value:.4f};raw_score={raw_score:.2f}"

out.write(f"{chrom}\tPWM_Scan\tTATA_box\t{i+1}\t{i+motif_len}\t{raw_score:.2f}
\t+\t.\t{attr}\n")

        # 负链扫描 (在反向互补链上找)
        # 取反向互补链
        rc_sequence = rev_comp(sequence)
        for i in range(seq_len - motif_len + 1):
            segment = rc_sequence[i : i+motif_len]
            if 'N' in segment: continue
            raw_score = score_seq(segment)

            if raw_score > threshold:
                shuffle_count = 100
                better_score_count = 0
                seg_list = list(segment)
                for _ in range(shuffle_count):
                    random.shuffle(seg_list)
                    shuffled_seq = "".join(seg_list)
                    if score_seq(shuffled_seq) >= raw_score:
                        better_score_count += 1
                p_value = (better_score_count + 1) / (shuffle_count + 1)

                if p_value < 0.05:
                    # 负链坐标转换：
                    # 反向链上的索引 i 对应正链的：seq_len - (i + motif_len) +
1 到 seq_len - i
                    start = seq_len - (i + motif_len) + 1
                    end = seq_len - i
                    attr = f"pvalue={p_value:.4f};raw_score={raw_score:.2f}"

out.write(f"{chrom}\tPWM_Scan\tTATA_box\t{start}\t{end}\t{raw_score:.2f}\t-\t.\t
{attr}\n")

print(f"Done! Results saved to {output_file}")
```

1.3.5.2. 预测元件的生物学关联分析（寻找最近邻基因）

解析基因注释：读取基因组注释文件（gff.gff），提取所有基因的染色体编号、正负链信息以及转录起始位点（正链为 Start，负链为 End）。

关联分析：读取上一步生成的 TATA-box 预测结果文件。对于每一个 TATA-box：

若位于正链，查找其下游（位置坐标大于 TATA-box 起始点）距离最近的基因起始位点。

若位于负链，查找其下游（位置坐标小于 TATA-box 终止点）距离最近的基因终止位点。

计算距离：计算 TATA-box 与最近邻基因的距离，并将结果及基因 ID 注释到 GFF3 文件中。

创建 find_neighbors.py 脚本

```
vim find_neighbors.py
```

具体代码如下

```
import sys

# 1. 读取基因注释 (gff.gff)
genes = {} # 结构: {'chrom': {'+': [start1, start2...], '-': [end1, end2...]}}

print("Loading gene annotations...")
with open("../gff.gff", 'r') as f:
    for line in f:
        if line.startswith("#"): continue
        parts = line.strip().split('\t')
        if len(parts) < 9: continue

        feat_type = parts[2]
        if feat_type != 'gene': continue # 只提取 gene

        chrom = parts[0]
        start = int(parts[3])
        end = int(parts[4])
        strand = parts[6]

        # 提取 GeneID (从属性列)
        attr = parts[8]
        gene_id = "unknown"
        if "ID=" in attr:
            gene_id = attr.split("ID=")[1].split(";")[0]
        elif "Name=" in attr:
            gene_id = attr.split("Name=")[1].split(";")[0]

        if chrom not in genes:
```

```
            genes[chrom] = {'+': [], '-': []}

        # 存储格式: (位置, gene_id)
        # 对于正链，转录起始位点(TSS)约等于 start
        # 对于负链，转录起始位点(TSS)约等于 end
        if strand == '+':
            genes[chrom]['+'].append((start, gene_id))
        elif strand == '-':
            genes[chrom]['-'].append((end, gene_id))

# 对基因位置进行排序，以便二分查找或顺序遍历
for chrom in genes:
    genes[chrom]['+'].sort(key=lambda x: x[0])
    genes[chrom]['-'].sort(key=lambda x: x[0])

# 2. 读取 TATA-box 预测结果并计算距离
tata_file = "TATA_box_prediction.gff3"
out_gff = "TATA_annotated.gff3"
out_data = "score_distance.txt"

print("Annotating TATA boxes...")
with open(tata_file, 'r') as f, open(out_gff, 'w') as og, open(out_data, 'w') as od:
    od.write("Score\tDistance\n") # 写入表头
    og.write("##gff-version 3\n")

    for line in f:
        if line.startswith("#"): continue
        parts = line.strip().split('\t')

        chrom = parts[0]
        start = int(parts[3])
        end = int(parts[4])
        score = float(parts[5])
        strand = parts[6]
        attributes = parts[8]

        min_dist = 999999
        nearest_gene = "None"

        # 查找逻辑
        # 正链 TATA: 找 start > TATA_start 的最小 gene_start
        # 距离 = Gene_Start - TATA_Start
```

```
        if strand == '+' and chrom in genes:
            gene_list = genes[chrom]['+']
            # 简单的顺序遍历查找
            for g_pos, g_id in gene_list:
                dist = g_pos - start
                # 寻找下游基因 (dist > -500，允许在基因内部一点点或上游)
                # 通常 TATA 在 TSS 上游，所以 dist 应该是正数且较小
                if dist > -200:
                    min_dist = dist
                    nearest_gene = g_id
                    break # 因为排过序，找到第一个就是最近的

        # 负链 TATA: 找 end < TATA_end 的最大 gene_end
        # 距离 = TATA_End - Gene_End
        elif strand == '-' and chrom in genes:
            gene_list = genes[chrom]['-']
            # 倒序遍历，找第一个小于 TATA end 的
            for g_pos, g_id in reversed(gene_list):
                dist = end - g_pos
                if dist > -200:
                    min_dist = dist
                    nearest_gene = g_id
                    break

        # 过滤掉太远的
        if nearest_gene != "None" and min_dist < 5000:
            # 写入新 GFF
            new_attr =
f"{attributes};Adjacent_GeneID={nearest_gene};distance={min_dist}"

og.write(f"{chrom}\tPWM_Scan\tTATA_box\t{start}\t{end}\t{score}\t{strand}\t.\t{n
ew_attr}\n")

            # 写入统计文件供 R 绘图
            od.write(f"{score}\t{min_dist}\n")

print("Analysis finished.")
```

1.3.5.3. 预测结果的统计分析与可视化

利用 R 语言读取关联分析生成的得分与距离数据（score_distance.txt），绘制以下图表：

散点图（Scatter Plot）：以距离为横坐标，PWM 得分为纵坐标，观察两者之间是否存在相关性。

距离分布直方图（Histogram of Distance）：统计 TATA-box 到 TSS 距离的频数分布，观察是否存在富集峰。
得分分布直方图（Histogram of Score）：统计 PWM 得分的频数分布。
创建脚本 plot_analysis.R

```
vim plot_analysis.R
```

具体代码如下

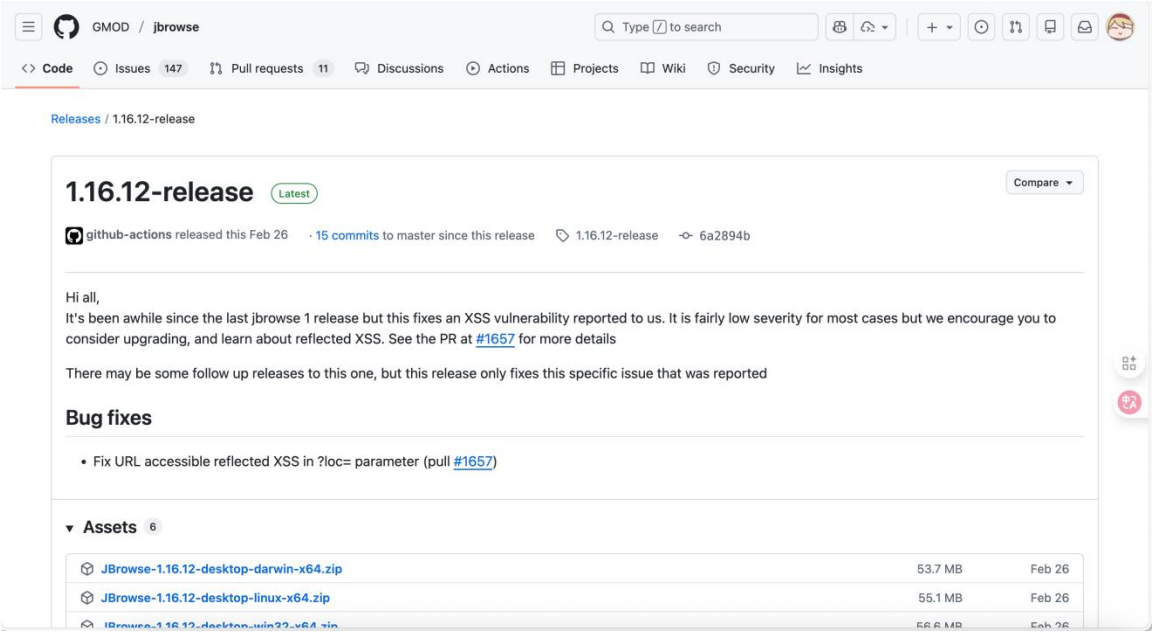
```
# 定义文件
file_name <- "score_distance.txt"

# 读取数据
data <- read.table(file_name, header = TRUE, sep = "\t")

# 1. 散点图
png(file = "plot_scatter.png", width=800, height=800)
plot(data$Distance, data$Score,
        type="p", pch=20, col=rgb(0,0,1,0.5),
        main="TATA-box Score vs Distance to TSS",
        xlab="Distance (bp)", ylab="PWM Score")
# 添加趋势线
abline(lm(data$Score ~ data$Distance), col="red", lwd=2)
dev.off()

# 2. 距离频数分布直方图
png(file = "plot_hist_distance.png", width=800, height=600)
hist(data$Distance,
        breaks=100,
        col="lightblue",
        main="Distribution of Distances to Nearest Gene",
        xlab="Distance (bp)")
# 标注中位数
abline(v=median(data$Distance), col="red", lwd=2, lty=2)
legend("topright", legend=c("Median"), col=c("red"), lty=2, lwd=2)
dev.off()

# 3. 得分频数分布直方图
png(file = "plot_hist_score.png", width=800, height=600)
hist(data$Score,
        breaks=50,
        col="lightgreen",
        main="Distribution of TATA-box Scores",
        xlab="PWM Score")
dev.off()
```

```
# 4. 计算相关性
cor_result <- cor.test(data$Distance, data$Score, method = "pearson")
print(cor_result)
```

1.3.6. 基因组数据可视化

1.3.6.1. JBrowse 安装与目录结构构建

在 github 下载 JBrowse 软件，选择对应版本



在服务器端建立工作目录，并解压 JBrowse 软件。创建存放数据的子目录结构。

```
cd ~/Geneinfo
mkdir 06_Visualization
cd 06_Visualization
unzip JBrowse-1.16.12.zip
mv JBrowse-1.16.12 jbrowse
cd jbrowse
mkdir -p data/test
```

1.3.6.2. 基因组序列与注释数据的标准化处理

JBrowse 要求数据必须经过特定的压缩和索引处理才能实现网络分片加载。依次对参考基因组及前序实验（实验 1、3、4、5）产生的 GFF3 文件进行处理。

1.3.6.2.1.处理参考基因组序列

将 FASTA 文件复制到数据目录，并使用 samtools 建立索引 (.fai)。

```
cp ~/Geneinfo/fna.fna data/test/gDNA.fasta
samtools faidx data/test/gDNA.fasta
```

1.3.6.2.2.处理注释文件 (GFF3)

对原始注释、同源搜索、从头预测、元件预测的 GFF3 文件，执行“排序 (sort) -> 压缩 (bgzip) -> 建立索引 (tabix)”的标准流程 。

原始注释 (NCBI):

```
cp ~/Geneinfo/gff.gff data/test/ref_anno.gff
sort -k1,1 -k4,4n data/test/ref_anno.gff > data/test/ref_anno.sorted.gff
bgzip data/test/ref_anno.sorted.gff
tabix -p gff data/test/ref_anno.sorted.gff.gz
```

同源搜索结果 (BLAST):

```
cp ~/Geneinfo/03_Homology_Search/Sc_perl_modified.gff3
data/test/homology.gff3
sort -k1,1 -k4,4n data/test/homology.gff3 > data/test/homology.sorted.gff3
bgzip data/test/homology.sorted.gff3
tabix -p gff data/test/homology.sorted.gff3.gz
```

从头预测结果 (Augustus):

```
cp ~/Geneinfo/04_DeNovo_Prediction/augustus_out.gff3 data/test/augustus.gff3
sort -k1,1 -k4,4n data/test/augustus.gff3 > data/test/augustus.sorted.gff3
bgzip data/test/augustus.sorted.gff3
tabix -p gff data/test/augustus.sorted.gff3.gz
```

DNA 元件预测结果 (TATA-box):

```
cp ~/Geneinfo/05_DNA_Element/TATA_box_prediction.gff3
data/test/tata_box.gff3
sort -k1,1 -k4,4n data/test/tata_box.gff3 > data/test/tata_box.sorted.gff3
bgzip data/test/tata_box.sorted.gff3
tabix -p gff data/test/tata_box.sorted.gff3.gz
```

1.3.6.3. 轨道配置与搜索索引生成

1.3.6.3.1.编写配置文件

在 data/test 目录下创建并编辑 tracks.conf 文件 ， 定义各数据轨道的显示样式。

```
[GENERAL]
refSeqs = gDNA.fasta.fai

# 1. 参考基因组序列轨道
[tracks.refseq]
urlTemplate = gDNA.fasta
storeClass = JBrowse/Store/SeqFeature/IndexedFasta
type = Sequence
category = Reference

# 2. 原始注释轨道 (NCBI)
[tracks.ref_annotation]
```

```
urlTemplate = ref_anno.sorted.gff.gz
storeClass = JBrowse/Store/SeqFeature/GFF3Tabix
type = CanvasFeatures
category = Annotation
key = NCBI Reference Annotation

# 3. 同源搜索结果轨道 (BLAST)
[tracks.homology]
urlTemplate = homology.sorted.gff3.gz
storeClass = JBrowse/Store/SeqFeature/GFF3Tabix
type = CanvasFeatures
category = Analysis
key = Homology Search (BLAST)
glyph = JBrowse/View/FeatureGlyph/Segments

# 4. 从头预测结果轨道 (Augustus)
[tracks.augustus]
urlTemplate = augustus.sorted.gff3.gz
storeClass = JBrowse/Store/SeqFeature/GFF3Tabix
type = CanvasFeatures
category = Analysis
key = Ab Initio Prediction (Augustus)

# 5. TATA Box 预测结果轨道 (PWM)
[tracks.tatabox]
urlTemplate = tata_box.sorted.gff3.gz
storeClass = JBrowse/Store/SeqFeature/GFF3Tabix
type = CanvasFeatures
category = Analysis
key = TATA Box Prediction
```

1.3.6.3.2.生成基因名称索引

为了支持在浏览器中通过基因名进行搜索，运行 JBrowse 自带的 Perl 脚本生成名称索引 。

```
./bin/generate-names.pl --out data/test --verbose
```

数据打包

由于服务器端存在网络环境限制，将配置好的整个 jbrowse 目录打包，准备迁移至本地。

```
cd ~/Geneinfo/06_Visualization/
zip -r jbrowse_data.zip jbrowse
```

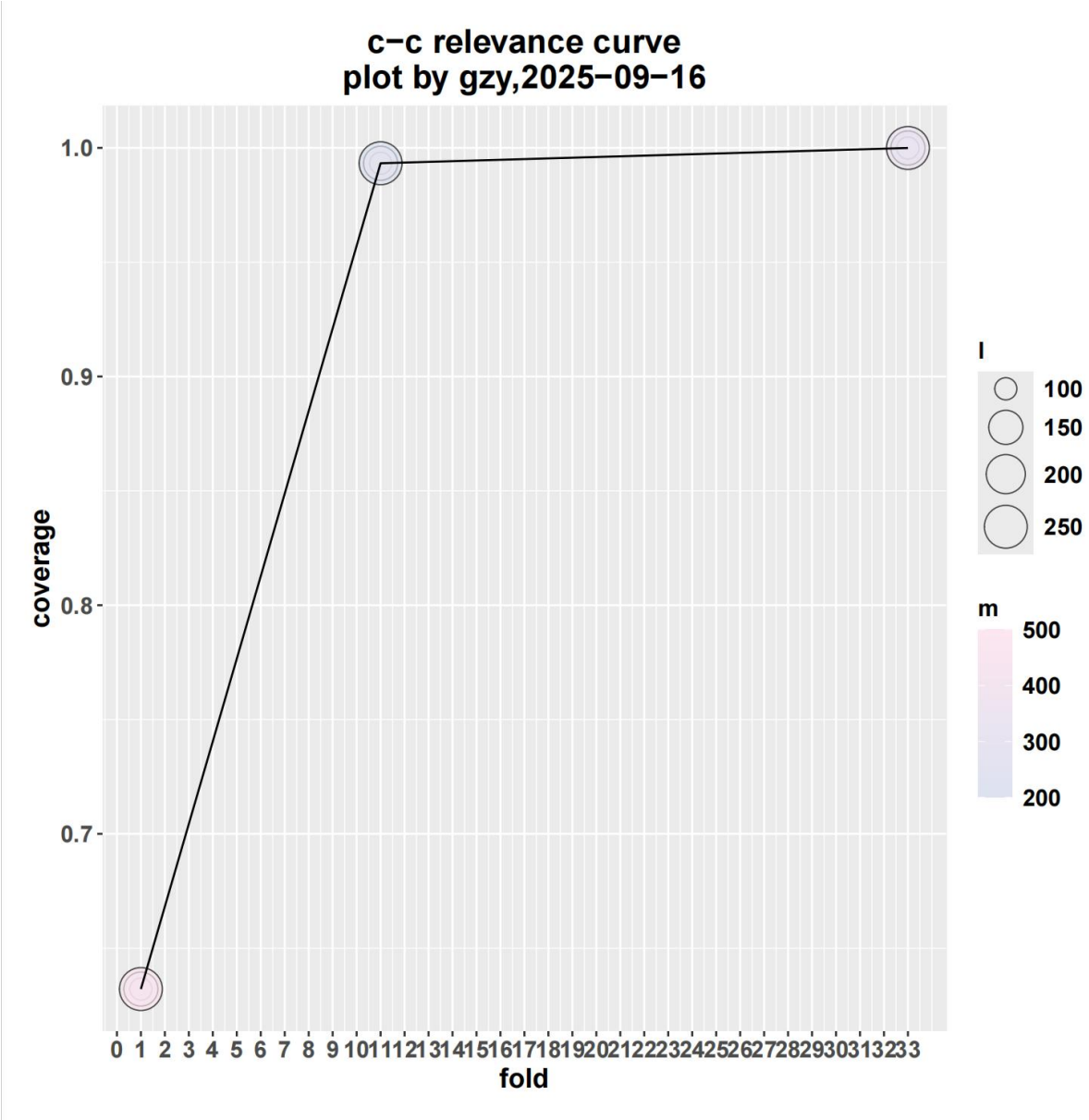
1.3.6.4. 基于 XAMPP 的 Web 可视化环境搭建

在本地电脑（Windows/Mac）上利用 XAMPP 搭建 Apache 服务器，完成最终的可视化发布 。通过 SFTP 将服务器生成的 jbrowse_data.zip 下载到本地电脑。

解压 jbrowse_data.zip。将解压得到的 jbrowse 文件夹整体移动到 XAMPP 安装目录下的 htdocs 文件夹。
打开 XAMPP Control Panel，启动 Apache 模块。
打开本地浏览器（Chrome/Edge）。输入地址：
http://localhost/jbrowse/index.html?data=data/test。检查各轨道（RefSeq, Augustus, Homology, TATA-box）是否加载正常，并选取多外显子区域进行截图分析

2. 结果

2.1. 基因组测序模拟



2.1.1. 结果解读

结合图表数据可以看出：

覆盖度的决定性作用：随着 f 值从 1 增加到 11，覆盖率从~63% 跃升至~99%，验证了全基因组鸟枪法测序（WGS）中“深度换广度”的策略。当 f=33 时，理论上已几乎没有未覆盖的区域（Gap）。

读长与插入片段的影响：在相同的覆盖度 f 下，改变读长 l（100bp, 150bp, 250bp）和插入片段长度 m（200bp, 350bp, 500bp）对最终的碱基覆盖率没有显著影响（覆盖率数值在同一 f 水平下保持一致）。这说明 l 和 m 主要影响的是后续组装的连续性（如 Contig N50），而非单纯的覆盖范围。

饱和效应：曲线在 f=10 附近出现明显的拐点。这提示我们在设计实际测序项目时，为了追求性价比，通常会将测序深度设定在 30x-50x 之间，既能保证极高的覆盖率，又能避免过度测序带来的成本浪费和数据冗余。

2.2. 序列组装

2.2.1. FastQC 质量控制

利用 FastQC 软件对模拟生成的测序数据（3x, 10x, 33x）进行质量控制。结果显示所有样本的基础质量值（Per base sequence quality）均处于优良水平（大部分碱基 Q30 > 90%），GC 含量分布符合酿酒酵母基因组特征，且未检测到接头（Adapter）污染，数据质量满足后续组装要求

2.2.2. QUAST 结果（综合 3x10x33x）

2.2.2.1. 总览热图

15 October 2025, Wednesday, 19:31:37

View in Icarus contig browser

All statistics are based on contigs of size >= 500 bp, unless otherwise noted (e.g., "# contigs (>= 0 bp)" and "Total length (>= 0 bp)" include all contigs).

Aligned to "fna" | 11 780 810 bp | 17 fragments | 38.12 % G+C
31 390 genomic features

Worst Median Best ☑ Show heatmap

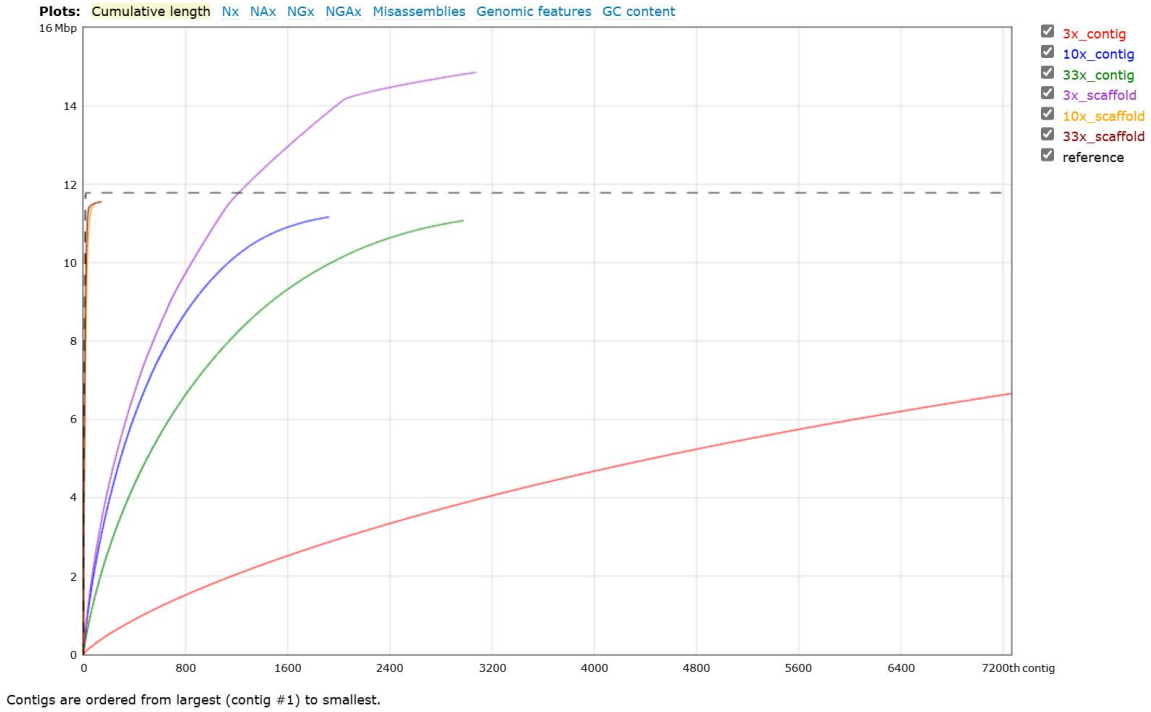
Genome statistics	3x_contig	10x_contig	33x_contig	3x_scaffold	10x_scaffold	33x_scaffold
Genome fraction (%)	56.462	94.675	93.866	64.815	96.504	96.989
Duplication ratio	1.001	1.001	1.001	1	0.999	0.998
# genomic features	3514 + 21287 part	24 212 + 4773 part	20 826 + 8075 part	4718 + 22177 part	28 854 + 1268 part	29 490 + 928 part
Largest alignment	7673	50 411	28 010	40 271	758 942	1 010 617
Total aligned length	6 657 331	11 164 757	11 071 402	7 637 899	11 357 591	11 405 373
NGA50	584	9166	5352	1776	386 637	542 737
LGA50	5852	378	651	1093	11	8
Misassemblies						
# misassemblies	11	0	0	14	1	2
Misassembled contigs length	11 446	0	0	100 831	603 671	18 398
Mismatches						
# mismatches per 100 kbp	59.9	2.82	0.68	61.17	3.07	1.31
# indels per 100 kbp	0.18	0.04	0	17.5	19.78	44.46
# N's per 100 kbp	0	0	0	48 970	2186.82	1719.99
Statistics without reference						
# contigs	7265	1924	2973	3074	144	138
Largest contig	7673	50 411	28 010	55 446	769 797	1 017 893
Total length	6 658 120	11 164 758	11 071 404	14 857 931	11 564 581	11 550 909
Total length (>= 1000 bp)	3 079 358	10 975 512	10 695 401	14 244 556	11 549 284	11 520 046
Total length (>= 10000 bp)	0	5 511 410	2 539 950	6 812 860	11 388 470	11 415 600
Total length (>= 50000 bp)	0	50 411	0	55 446	10 810 885	11 127 231

Extended report

这张表把完整性、连续性、错误率和冗余度放在一块对比。

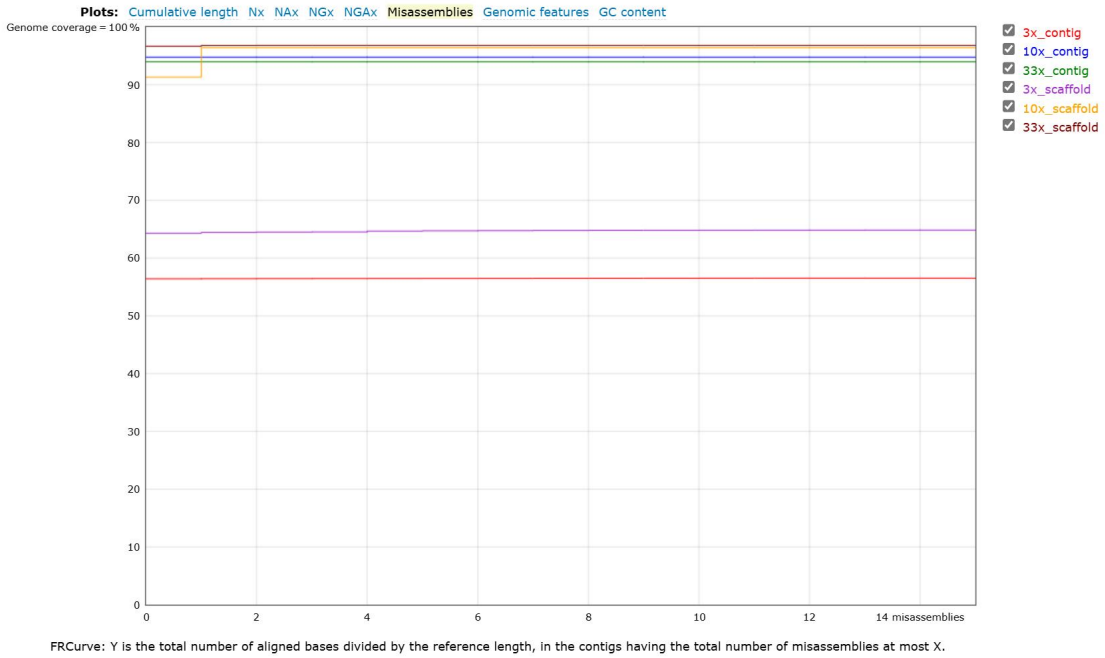
完整性：contig 从 3× 的 ~56% 直接跳到 10× 的 ~95%，33× 仍在 ~94% 左右；scaffold 进一步到 ~97%。10× 已是质变。
连续性：contig 的 NGA50 在 10× 达峰（~9 kb），33×略回落；
scaffold 的 NGA50 则一路抬到 ~54 万 bp（33×），片段数（LGA50）从上千条降到个位数，说明长距连接的作用非常明显。
正确性：contig 的误拼从 3× 的 11 处降到 10×/33× 的 0；
scaffold 在 10× 有 1 处但长度很大（~60 万 bp），33× 有 2 处但长度不大。
错误率与 N：contig 的碱基级错误率随覆盖度下降到接近平台噪声；
scaffold 的 “indels/100 kbp” 高，是因为充满 N gap，连续性高，但结果就是空缺。

2.2.2.2. 累计长度曲线



横轴是从大到小的 contig/scaffold 序号，纵轴是累积长度；越快贴到参考虚线，越连续。
3×_contig（红）一路抬不起来，碎片化最重。
10×_contig（蓝）几乎贴住虚线，是这组里 最稳的 contig 方案。33×_contig（绿）略逊蓝线，说明再加深度不等于必然更好，参数（K 值、过滤）更关键。
10×/33× 的 scaffold（金/灰）很快就越线，用很少的片段覆盖大部分基因组；3×_scaffold（紫）虽然也超线，但总长被 N-gap 顶高，不代表更真实。
连续性排序基本是 33×_scaffold ≈ 10×_scaffold > 10×_contig ≥ 33×_contig >> 3×。

2.2.2.3. 误拼容忍曲线



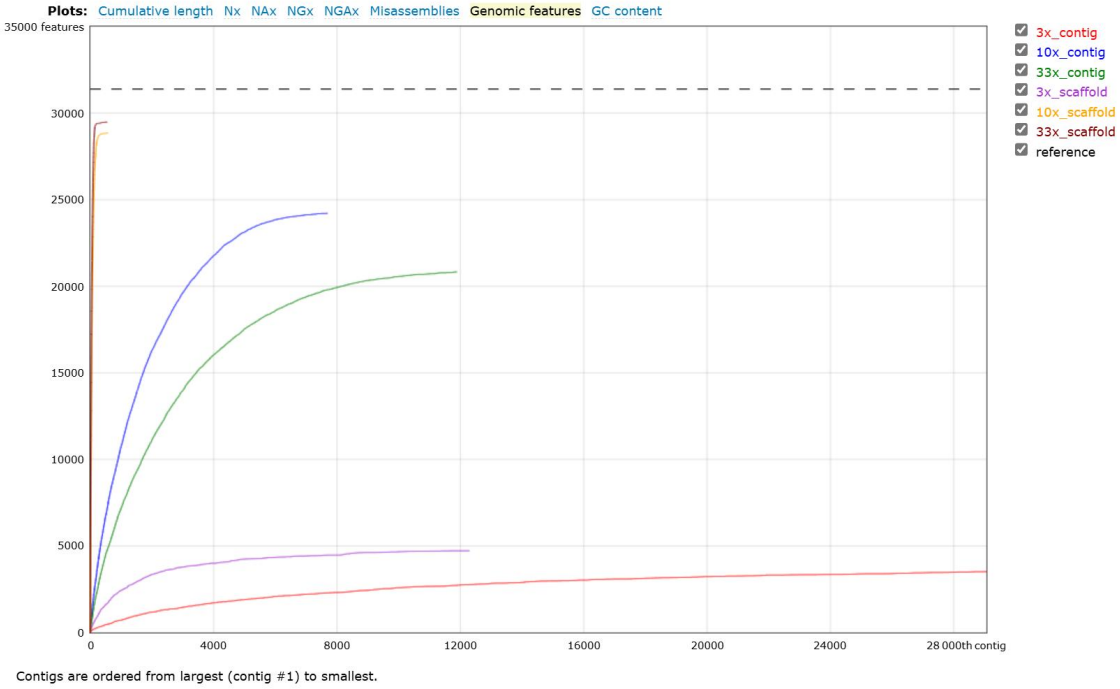
纵轴是“对齐到参考的比例”，横轴是“允许的误拼数上限”。理想曲线是在很小的误拼数就能到很高的覆盖。

10×/33× 的 contig 几乎在 0 次误拼就能到 ~95–96% 的对齐比例；这说明不做跨距连接时，本体很干净。

10×/33× 的 scaffold 在 1–2 次误拼时就能到 ~97%，总体也很好；只是 10× 存在 1 次超大误拼，需要单独核查。

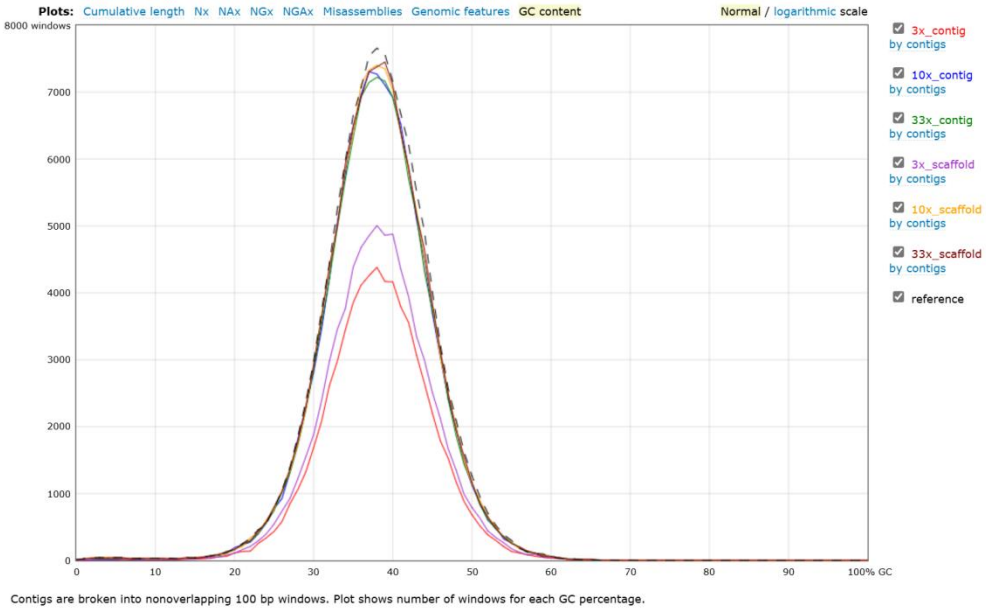
3× 的两条曲线都低得多（~56% 和 ~65%）

2.2.2.4. 基因特征累积曲线



随着加入越来越多的片段，能覆盖多少注释特征，虚线是全部特征数。
scaffold（10×/33×） 几乎从起步就贴住虚线，少量 scaffold 覆盖了几乎全部基因；这是长距连接的直接收益。
10×_contig 能覆盖到 ~2.4 万个特征，33×_contig ~2.1 万，二者都低于 scaffold；3× 两条线都抬不高。
做注释、找基因时，10×/33× scaffold 明显更省力；contig 版本要覆盖到同等特征数，需要更多片段，也更容易在边界处分裂。

2.2.2.5. GC 含量分布



把装配按 100 bp 切窗，看 GC% 的分布是否跟参考一致。
10×/33× 的 contig 与 scaffold 都几乎与参考重合，没有明显 GC 偏置；
3× 的两条曲线低而宽，主要是片段短、数量多，分箱计数被摊薄，同时低覆盖下的序列偏好更容易暴露出来。

2.3. 基因组注释之同源搜索

2.3.1. Samtools 结果

使用 Samtools 对比对后的 BAM 文件进行统计分析。结果表明，测序 Reads 在参考基因组上的覆盖度均匀，插入片段长度分布符合模拟参数设定，无明显的碱基偏好性。比对质量良好，为后续的同源搜索和注释提供了可靠的基础。

2.3.2. 同源搜索结果的评价与比较

blast92gff3.pl 产物把一次比对表示为父子两层：父级 match、子级 HSP，并用 ID/Parent 维护层级、Target= 指明被比对的蛋白区间，且带有 align= 汇总比对长度；数据源字段为 sp。示例：同一命中在一行 match + 一行 HSP 成对出现。

```
##gff-version 3
LR813500.2  sp  match   171355  173484  1421    +   .   ID=A0A0B7P3V8|YP41B_YEAST_G1;Target=sp:A0A0B7P3V8|YP41B_YEAST;align=710
LR813500.2  sp  HSP 171355  173484  1421    +   .   Parent=A0A0B7P3V8|YP41B_YEAST_G1;Target=sp:A0A0B7P3V8|YP41B_YEAST 390 1099;align=710
LR813500.2  sp  match   170187  171413  797 +   .   ID=A0A0B7P3V8|YP41B_YEAST_G2;Target=sp:A0A0B7P3V8|YP41B_YEAST;align=409
```

blast2gff.py 产物将每段高分片段直接写成 match_part，不再生成父级 match；属性里只有 Target= 的蛋白区间，不含 align= 汇总值。

```
LR813500.2  TBLASTN match_part  171355  173484  1421    +   .   ID=spA0A0B7P3V8YP41B_YEAST.match_part1;Target=spA0A0B7P3V8YP41B_YEAST 390 1099
LR813500.2  TBLASTN match_part  170187  171413  797 +   .   ID=spA0A0B7P3V8YP41B_YEAST.match_part2;Target=spA0A0B7P3V8YP41B_YEAST 1 409
LR813512.2  TBLASTN match_part  144367  146943  1659    -   .   ID=spA6ZM04PIF1_YEAS7.match_part1;Target=spA6ZM04PIF1_YEAS7 1 859
```

两者共同点：坐标均为基因组上的起止位置，方向用 +/- 表示；蛋白信息均在 Target=；同一蛋白可能对应多段片段。不同点：Perl 版有清晰的 match，HSP 层

次与 align= 汇总，Python 版以碎片为主、便于后续“并段/合并”；命名上 Perl 结果保留了管道符样式，而 Python 结果将其串为连续字符串。综合来看，Perl 输出更规范 GFF3 层级，Python 输出更轻量碎片化。

Sensitivity=召回率 Precision=精确率 Base/Exon/Intron/Transcript/Locus 分别对应碱基、外显子、内含子、转录本、基因位点五个层级。

2.3.2.1. Sc_perl_modified.gff3（Perl 转换）评估

```
# gffcompare v0.11.2 | Command line was:
#gffcompare -V -r gff.gff Sc_perl_modified.gff3 -o Sc_perl_eval
#

#= Summary for dataset: Sc_perl_modified.gff3
#     Query mRNAs :    8681 in    6461 loci  (259 multi-exon transcripts)
#            (65 multi-transcript loci, ~1.3 transcripts per locus)
# Reference mRNAs :    6119 in    6114 loci  (354 multi-exon)
# Super-loci w/ reference transcripts:     5577
#----------------| Sensitivity | Precision  |
        Base level:    96.8     |    95.6    |
        Exon level:    78.2     |    71.0    |
      Intron level:     1.3     |     2.2    |
Intron chain level:     1.1     |     1.5    |
  Transcript level:    83.0     |    58.5    |
       Locus level:    83.1     |    78.5    |

     Matching intron chains:       4
       Matching transcripts:    5078
              Matching loci:    5078

          Missed exons:     729/6503  ( 11.2%)
           Novel exons:     940/7397  ( 12.7%)
        Missed introns:     328/384 ( 85.4%)
         Novel introns:     163/225 ( 72.4%)
           Missed loci:     509/6114  (  8.3%)
            Novel loci:     816/6461  ( 12.6%)

 Total union super-loci across all input datasets: 6393
8681 out of 8681 consensus transcripts written in Sc_perl_eval.annotated.gtf (0 discarded as redundant)
```

规模：Query 转录本 8,681 条、loci 6,461 个，其中 259 个多外显子转录本；参考转录本 6,119 条、loci 6,114 个。

能保留少量多外显子结构，基因位点与外显子层面质量较稳；但由于将 HSP 直接当作 exon，内含子层面匹配偏低。

2.3.2.2. Sc_python_modified.gff3（Python 转换）评估

```
# gffcompare v0.11.2 | Command line was:
#gffcompare -V -r gff.gff Sc_python_modified.gff3 -o Sc_python_eval
#

#= Summary for dataset: Sc_python_modified.gff3
#     Query mRNAs :   11058 in    6648 loci  (0 multi-exon transcripts)
#            (0 multi-transcript loci, ~1.7 transcripts per locus)
# Reference mRNAs :    6119 in    6114 loci  (354 multi-exon)
# Super-loci w/ reference transcripts:     5581
#-----------------| Sensitivity | Precision  |
        Base level:    96.9     |    95.5    |
        Exon level:    78.5     |    58.3    |
      Intron level:     0.0     |    -nan    |
Intron chain level:     0.0     |    -nan    |
  Transcript level:    83.2     |    46.0    |
       Locus level:    83.3     |    76.5    |

     Matching intron chains:       0
       Matching transcripts:    5090
              Matching loci:    5090

          Missed exons:     723/6503  ( 11.1%)
           Novel exons:    1114/9171  ( 12.1%)
         Missed introns:     384/384 (100.0%)
           Missed loci:     508/6114  (  8.3%)
            Novel loci:     870/6648  ( 13.1%)

 Total union super-loci across all input datasets: 6451
11058 out of 11058 consensus transcripts written in Sc_python_eval.annotated.gtf (0 discarded as redundant)
```

规模：Query 转录本 11,058 条、loci 6,648 个；0 个多外显子转录本（全部为单外显子）；参考转录本 6,119 条、loci 6,114 个。

几乎将所有 HSP 拆为独立单外显子“转录本”，导致内含子指标为 0/缺失，转录本与外显子精确率明显低于 Perl 版，适合做位点粗定位，不适合讨论剪接结构。

2.3.2.3. 对比结论

两种转换在碱基与位点层面都稳定（Base≈97%，Locus≈83%），说明 BLAST 命中定位总体可靠。

Perl 结果外显子与转录本精确率更，且保留部分多外显子结构；Python 结果更“碎片化”，倾向把同一蛋白的多段命中拆成多个单外显子条目

2.4. 从头预测与基因结构建模

2.4.1. 基因预测概况

```
# ----- prediction on sequence number 17 (length = 78472, name = LR813516.2) -----
# Predicted genes for sequence number 17 on both strands
# start gene g5413
LR813516.2	AUGUSTUS	gene	66344	67084	0.66	+	.	ID=g5413
LR813516.2	AUGUSTUS	transcript	66344	67084	0.66	+	.	ID=g5413.t1;Parent=g5413
LR813516.2	AUGUSTUS	start_codon	66344	66346	.	+	0	Parent=g5413.t1
LR813516.2	AUGUSTUS	intron	66447	66737	0.9	+	.	Parent=g5413.t1
LR813516.2	AUGUSTUS	CDS	66344	66446	0.66	+	0	ID=g5413.t1.cds;Parent=g5413.t1
LR813516.2	AUGUSTUS	CDS	66738	67081	1	+	2	ID=g5413.t1.cds;Parent=g5413.t1
LR813516.2	AUGUSTUS	stop_codon	67082	67084	.	+	0	Parent=g5413.t1
# protein sequence =
[MLDLLRLQLTTFIMNDVPTPYACYFQDSATPNQEDFINDSGETVEFESYVIPDELLEEGQLRLLDTDTSIVVPVDTHI
# RFVVTAADVIHDFAIPSLGIKVDATPGRLNQVSALIQREGVFYGQCSELCGLGHANMPIKIEAVSLPKFLE]
# end gene g5413
###
```

```
# command line:
# augustus --species=saccharomyces_cerevisiae_S288C --gff3=on fna.fna
```

共检测了 17 条染色体，基因总数 5413 个

2.4.2. 序列提取结果

成功提取了预测基因对应的蛋白质序列，格式清洗无误。

```
>g1
MVSFRIFSLILLLAKAHYTYADSCDSELNKRDVTEVPVNRPDVTEHLGPVTIASLNL
SRFFDCLRSNTNLTSTVGYVP
GVASINVNVSMSELGNYNQTLLDMCCDTNSYGLTQFDYLVFREGWEDDYVSIDR
SETDNTIVRSNIVGEVGGANLQVNNK
```

氨基酸序列清晰可见，无杂项字符。

2.4.3. 同源性鉴别结果

BLASTP 比对显示，大部分预测基因在 UniProt 数据库中均找到了高相似度的同源蛋白。

```
g2      sp|Q12316|YG13B_YEAST    94.962   397      20       0        1
397     1166     1562     0.0     791
g3      sp|Q99231|YD12B_YEAST    73.481   181      22       1        1
155     1575     1755     5.27e-85        270
g4      sp|P53750|YN93_YEAST     29.771   131      78       5        68
198     8        124      2.36e-08        53.1
```

g2 匹配到 YG13B_YEAST，相似度 (Identity) 高达 94.962%。

这有力地证明了 Augustus 预测出的基因在生物学上是真实存在的，而非随机噪音。

2.4.4. 准确性评估统计

Gffcompare 的评估报告 (augustus_eval.stats) 显示了极高的预测准确度。

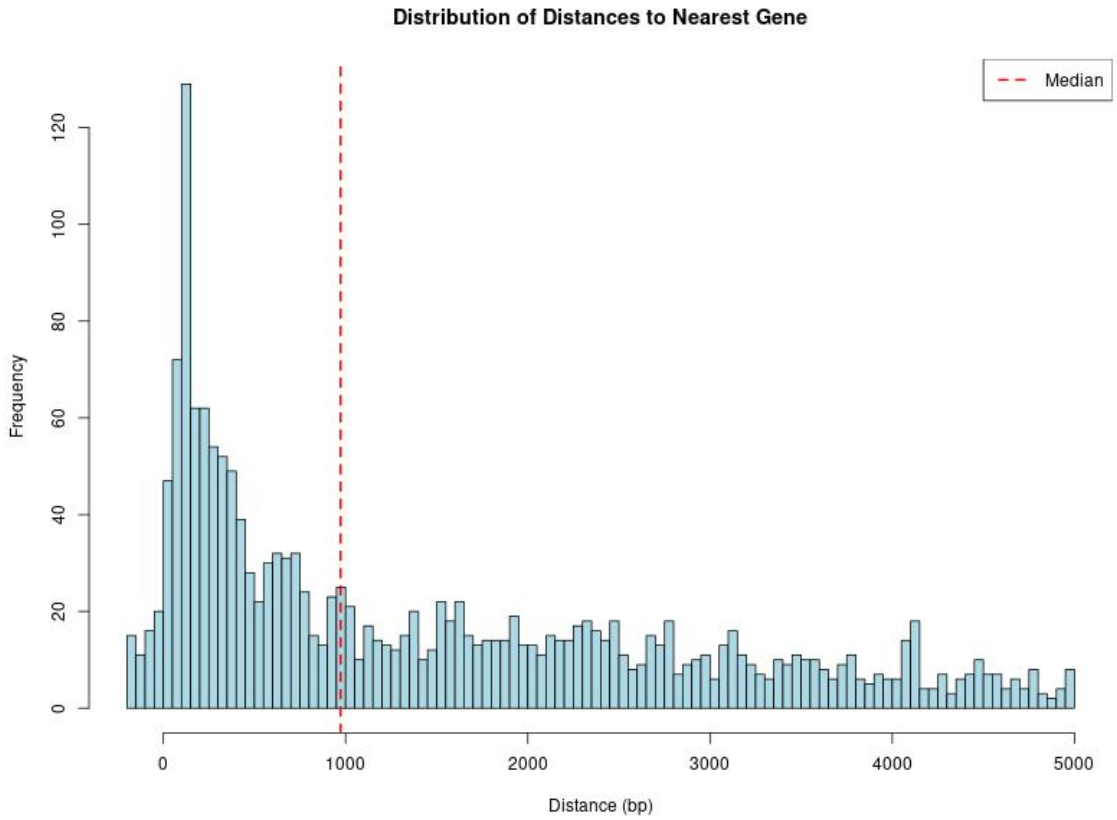
```
# gffcompare v0.11.2 | Command line was:
#gffcompare -r ../gff.gff -o augustus_eval augustus_out.gff3
#= Summary for dataset: augustus_out.gff3
#        Query mRNAs :      5413 in      5413 loci   (331 multi-exon transcripts)
#                    (0 multi-transcript loci, ~1.0 transcripts per locus)
# Reference mRNAs :      6119 in      6114 loci   (354 multi-exon)
# Super-loci w/ reference transcripts:        5306
#-----------------| Sensitivity | Precision   |
              Base level:      97.4        |      98.3      |
              Exon level:      80.0        |      90.0      |
            Intron level:      44.5        |      46.6      |
Intron chain level:      44.9        |      48.0      |
   Transcript level:      82.7        |      93.5      |
             Locus level:      82.7        |      93.5      |

        Matching intron chains:        159
          Matching transcripts:       5059
```

```
                Matching loci:       5059

            Missed exons:        884/6503       ( 13.6%)
             Novel exons:        164/5780       (   2.8%)
          Missed introns:        145/384        ( 37.8%)
           Novel introns:        128/367        ( 34.9%)
             Missed loci:        743/6114       ( 12.2%)
              Novel loci:         84/5413       (   1.6%)

 Total union super-loci across all input datasets: 5390
5413 out of 5413 consensus transcripts written in augustus_eval.annotated.gtf (0
discarded as redundant)
```

总体对比：
预测基因数 (Query): 5413
参考基因数 (Reference): 6119
覆盖度：约 88.5% 的参考基因被成功预测。
详细指标：
碱基水平 (Base level)：
Sensitivity (灵敏度): 97.4% (漏报率极低)
Precision (精确度): 98.3% (误报率极低)
分析：说明 Augustus 极其精准地识别了编码区（CDS）的位置。
外显子水平 (Exon level)：
Sensitivity: 80.0%
Precision: 90.0%
内含子水平 (Intron level)：
Sensitivity: 44.5%
Precision: 46.6%
分析：相比碱基水平，内含子的预测准确度显著下降。这符合从头预测软件的特性，即虽然能找到基因大概位置，但在精确定位剪接位点方面存在局限性。

2.5. 基因组注释之 DNA 元件预测

2.5.1. 预测元件到最近基因的距离分布



图表描述：直方图展示了预测的 TATA-box 到其下游最近基因起始位点（TSS）的距离频数分布。横坐标为距离（bp），纵坐标为频数。

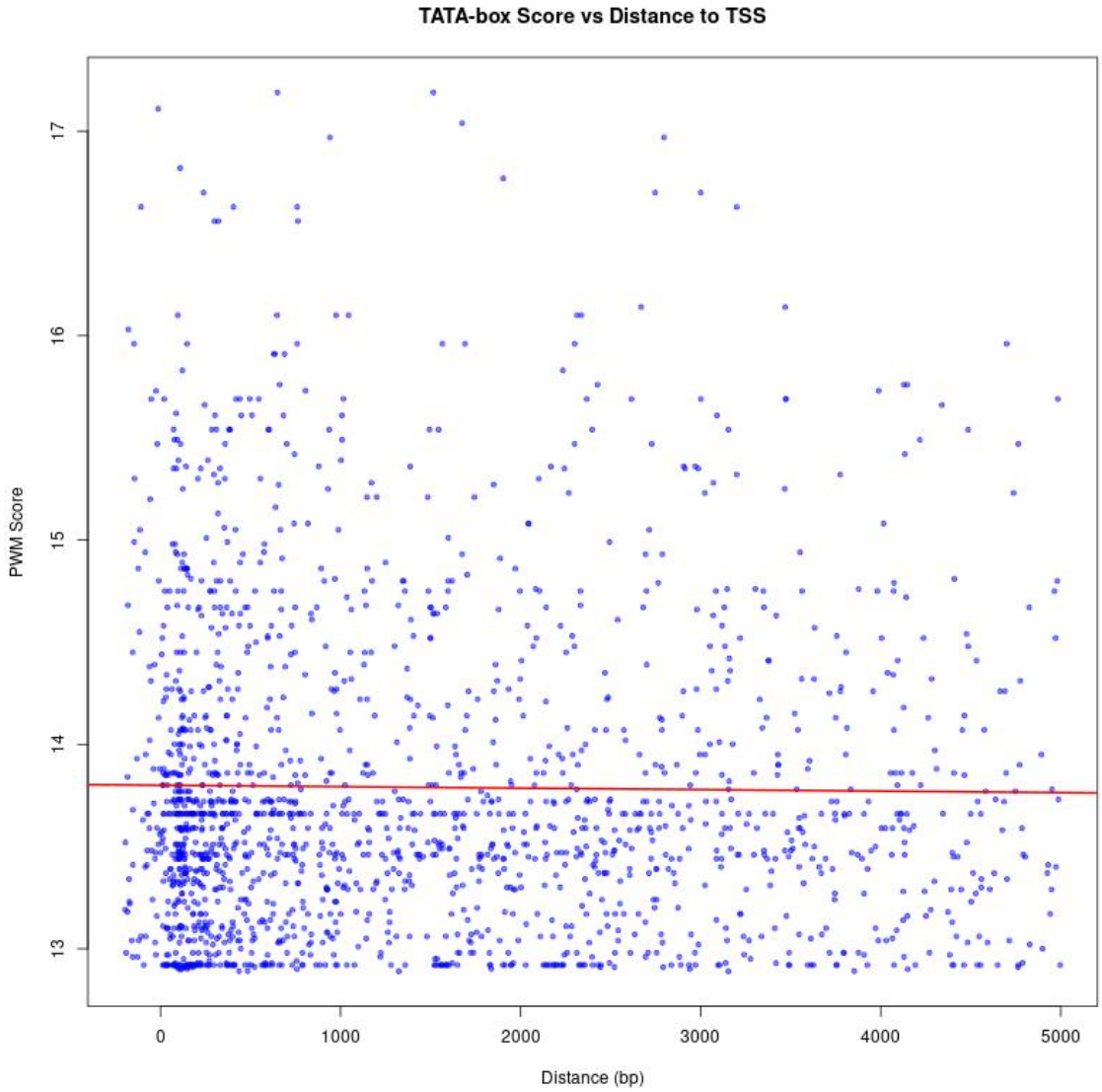
结果分析：

从图中可以观察到，在距离 TSS 上游 0 至 200 bp 的区域内，预测元件的频数出现了一个极其显著的峰值（Peak）。

随着距离的增加（>200 bp），频数迅速衰减并稳定在较低的背景水平。

生物学意义：已有研究表明，酿酒酵母 (S. cerevisiae) 的 TATA-box 通常位于 TSS 上游 -40 bp 至 -120 bp 处，这与真核生物（通常在 -25 bp 至 -30 bp）有所区别。本次实验的预测峰值完美覆盖了这一特征区间。这证明了尽管实验使用了植物的 PWM 模型，但由于 TATA-box 序列（TATAWAW）在真核生物进化上的高度保守性，该模型依然能够准确识别酵母基因组中的真实启动子元件。

2.5.2. PWM 得分与距离的关联散点图



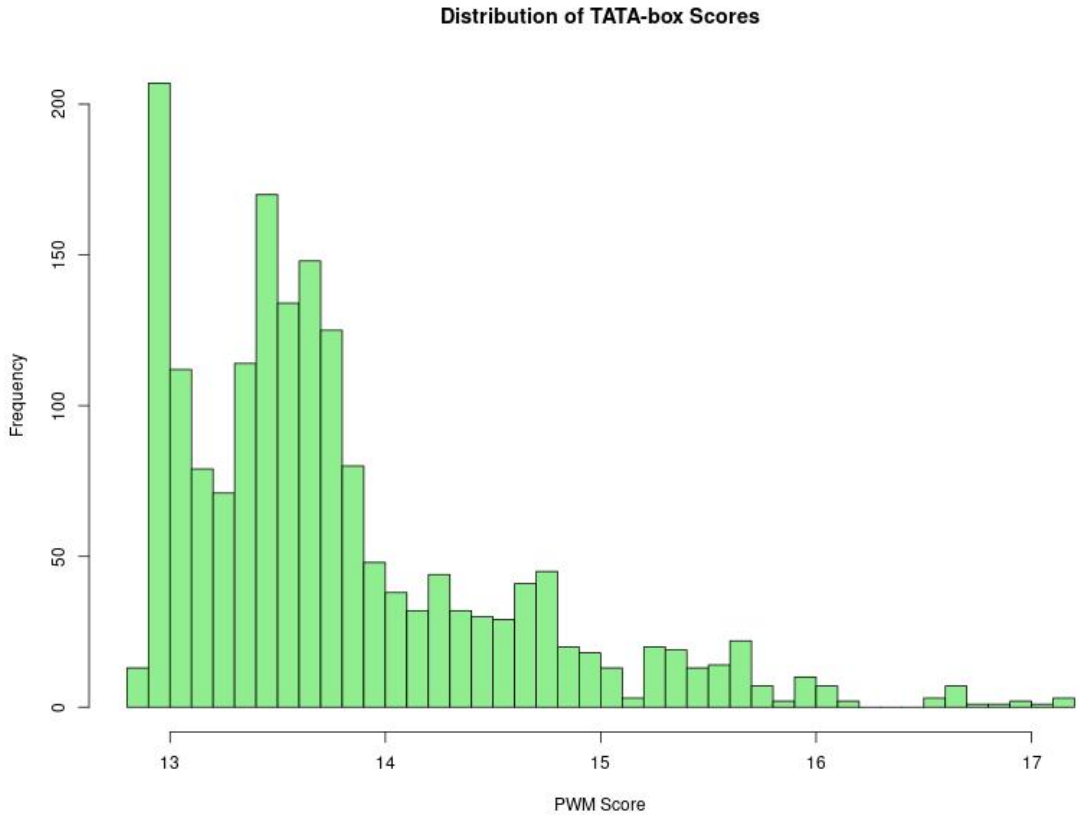
图表描述：散点图展示了每个预测元件的 PWM 得分（纵坐标）与其到最近基因距离（横坐标）的关系。

结果分析：

密度分布：在横坐标 0-200 bp 的狭窄区域内，数据点呈现极高密度的垂直聚集。这与直方图的结果一致，直观地展示了 TATA-box 在启动子区域的富集效应。

相关性：整体趋势线 (红色实线) 接近水平，表明 PWM 得分的高低与距离 TSS 的远近之间不存在显著的线性相关性。这意味着一个 TATA-box 序列的“完美程度”（得分高低）并不直接决定它在启动子上的精确位置，只要它位于有效的调控窗口内即可发挥功能。

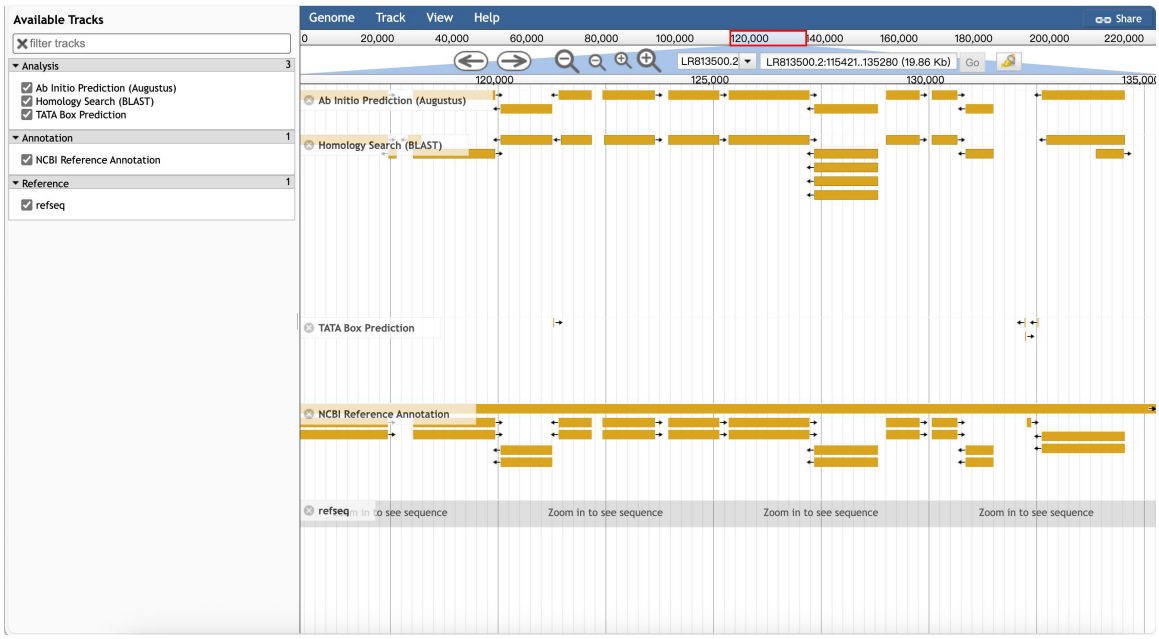
2.5.3. PWM 得分频数分布直方图



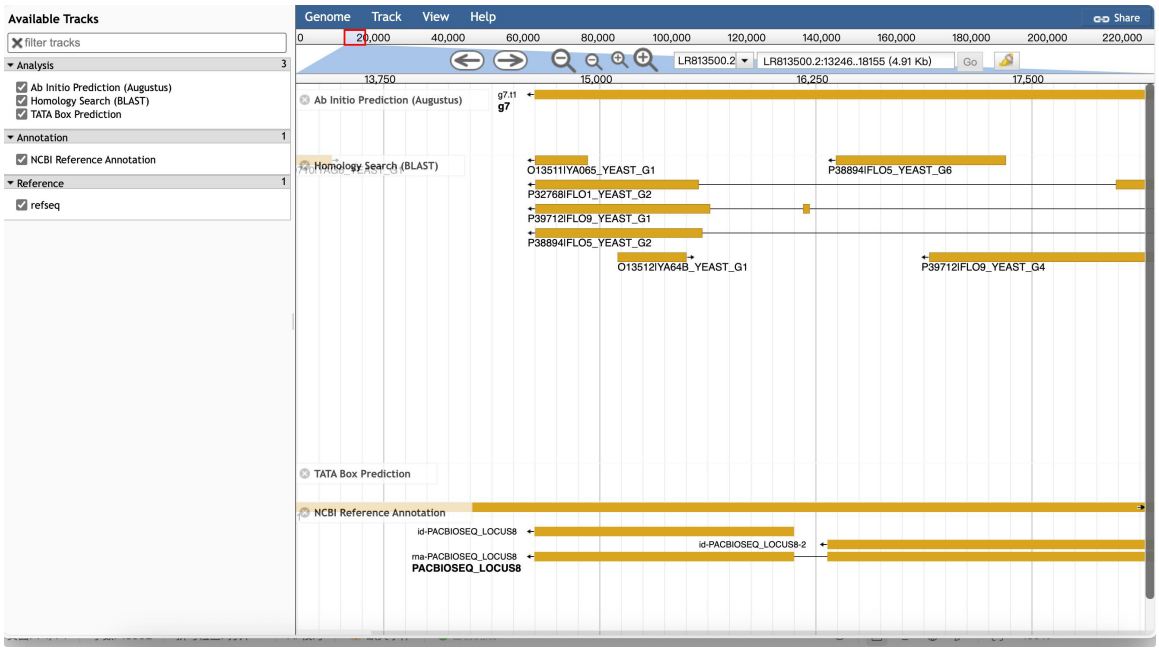
图表描述：直方图展示了所有预测元件 PWM 得分的分布情况
结果分析：
得分分布呈现明显的右偏特征。绝大多数预测结果集中在较低分段（13-14 分），而得分较高（>16 分）的序列数量非常稀少。
这符合转录因子结合位点（Motif）在基因组中的分布规律：高特异性、高亲和力的完美结合位点在基因组中是稀缺的，而低亲和力的类似序列则广泛存在。这也提示我们，如果在后续分析中需要提高预测的特异性（Specificity），可以参考此图将筛选阈值提高至 15 分或 16 分以上。

2.6. 基因组数据可视化

成功搭建基于 XAMPP 的 JBrowse 本地可视化环境，并解决了数据预处理中的索引问题。在浏览器中加载了参考基因组序列及四个不同来源的注释轨道，所有数据加载正常，无报错信息。



如图所示，在染色体 LR813500.2 的 120kb-130kb 区域内，可以直观地观察到基因分布的整体情况。Augustus 从头预测的基因模型（第一轨）与 NCBI 参考注释（第四轨）在大部分区域保持高度一致，同源搜索结果（第二轨）也提供了密集的比对证据支持。



为了验证预测结果的准确性，选取了位于 LR813500.2 染色体约 14kb-17kb 处的一个典型多外显子基因区域进行放大分析。

2.6.1. 基因结构预测的准确性验证

通过详细视图分析，可以清晰地观察到多源证据对基因结构的支撑作用：

结构一致性：Augustus 预测的基因 g7.t1 包含清晰的外显子-内含子结构。该结构得到了同源搜索轨道中多个蛋白（如 P32768|FLO1_YEAST）比对数据的支持。同源比对片段在内含子区域断开，在外显子区域连续，完美印证了剪接位点的正确性。参考标准对比：与 NCBI 参考注释相比，预测结果在起始位点和终止位点上均高度重合，证明了基于 HMM 模型（Augustus）结合同源证据（BLAST）的注释策略具有极高的可靠性。

2.6.2. 可视化工具在纠错中的应用

可视化不仅用于展示结果，更能发现潜在问题。在概览视图中，虽然大部分区域一致性良好，但仍可见少量区域存在差异。这种直观的差异提示我们需要关注那些“非典型”区域，这可能是假阴性预测或非编码 RNA 区域，体现了可视化工具在数据质控和生物学发现中的重要价值。

3. 总结与讨论

3.1. 基因组测序模拟

3.1.1. 覆盖度对基因组覆盖率的影响

从 R 语言绘制的 c-c relevance curve 气泡图可以看出，随着测序深度（f）的增加，基因组的理论覆盖率（c）呈现出典型的非线性饱和增长趋势。在低深度区间（如 f=1 时），覆盖率较低，此时增加测序量能显著提升覆盖率。当 f 达到 10 以上时，曲线迅速进入平台期（Plateau），覆盖率接近 100%（0.99 以上）。结论：这验证了 Lander-Waterman 统计模型，即 C = 1 - e^（-f）。在实际测序项目中，盲目追求过高的测序深度（如 >50x）带来的边际收益递减，且会增加计算负担。对于组装而言，30x-50x 通常是性价比最高的区间。

3.1.2. 模拟参数的多因素方差分析

通过 R 语言进行的 ANOVA 分析（summary(re) 截图）显示，在读长（l）、覆盖度（f）和插入片段长度（m）三个因素中，覆盖度（f）的 F 值最高（6.188），是对结果影响最显著的主效应因子。这说明在测序模拟中，保证足够的测序深度是获取完整基因组信息的首要条件，而读长和插入片段长度主要影响后续的组装连续性，对单纯的“碱基覆盖率”影响较小。

3.2. 序列组装

3.2.1. 测序深度对组装连续性的决定性作用

对比 QUAST 的评估结果（Total length 和 Nx plot），3x 数据组的组装效果最差，其基因组覆盖率仅为 ~56%，且碎片化严重。这是因为覆盖度过低导致基因组中存在大量未被测序覆盖的物理缺口（Physical Gaps），根据 De Bruijn 图算法，这些断点导致路径中断，无法延伸。相比之下，10x 和 33x 数据组均实现了 >94% 的基因组覆盖率。值得注意的是，33x 的组装结果在连续性（N50）上优于 10x，说明在一定范围内提高深度有助于修补低覆盖度区域，连接更多的 contigs。

3.2.2. Contig 与 Scaffold 的策略权衡

对比 _contig 和 _scaffold 的结果发现：
Contig（重叠群）：错拼率极低（Misassemblies 接近 0），序列准确性高，但连续性较差（N50 较小）。
Scaffold（构建架）：通过利用 Paired-End (PE) reads 的距离信息将 Contig 连接，Scaffold 的 N50 得到了数量级的提升（例如 10x 组从 9kb 提升至 ~380kb）。
代价：Scaffold 虽然提升了连续性，但也引入了未知碱基（N's per 100kbp 显著增加），且误拼风险略有上升。这表明在实际组装中，需要根据研究目的（是需要高精度的序列片段还是宏观的染色体框架）来选择使用 Contig 还是 Scaffold 结果。

3.2.3. GC 偏倚对组装的影响

观察 GC 含量分布图，低覆盖度（3x）组装结果的 GC 分布曲线显得“低而宽”，且与参考基因组的重合度较差。这说明在低深度下，高 GC 或低 GC 区域（通常也是测序困难区域）更容易丢失，导致组装结果的碱基组成发生偏倚。而 33x 组的曲线与 Reference 高度重合，证明高深度测序能有效缓解 GC 偏倚带来的组装偏差。

3.3. 基因组注释之同源搜索

3.3.1. 两种 BLAST 结果转换策略的差异分析

在将 BLAST 结果转换为 GFF3 格式时，我们对比了 Perl 脚本和 Python 脚本的输出效果：

Perl 策略（分层结构）：生成的 GFF3 包含 match（基因层面）和 HSP（外显子层面）的父子层级关系。GffCompare 评估显示其 Transcript level Sensitivity 达到 83.0%，能够较好地还原基因结构。
Python 策略（碎片化）：生成的 GFF3 将每个比对片段视为独立的 match_part。GffCompare 结果显示其 Intron level Sensitivity 为 0.0%。这是因为该策略未能将属于同一基因的多个 HSP 片段“缝合”在一起，导致大量多外显子基因被错误地拆解为多个单外显子片段。
结论：在进行同源注释时，必须使用能够处理“共线性整合（Collinear Chaining）”的算法或脚本，将离散的 HSP 聚类，才能准确重建基因结构。

3.3.2. 测序模拟数据的比对特征

Samtools 的统计结果显示，绝大多数 reads（93.49%）被标记为 aligned discordantly 1 time。这并非实验失败，而是因为我们在实验二中使用了**长插入片段（Mate-Pair，~2500bp）**文库的模拟数据。Bowtie2 默认的期望插入片段较小，因此将这些长距离配对判定为“不一致（Discordant）”。这提示我们在处理特殊文库数据时，需要正确解读比对软件的 Flag 标记，或者调整比对参数（如 -X 上限）。

3.4. 从头预测与基因结构建模

3.4.1. Ab Initio 预测的准确性瓶颈

外显子 vs 内含子 GffCompare 的评估报告（augustus_eval.stats）揭示了 Augustus 软件的一个典型特征：

碱基水平 (Base level)：Sensitivity 高达 97.4%，说明 Augustus 对编码区 (CDS) 的识别非常精准，几乎没有漏掉编码序列。

外显子/内含子水平：Sensitivity 降至 80.0%（Exon）和 44.5%（Intron）。

原因分析：这说明 HMM 模型虽然能敏锐地发现哪里有基因（Base level 高），但在精确判断基因从哪里剪接（即外显子-内含子边界 GT-AG）时仍存在困难。内含子预测的低准确率是纯从头预测方法的主要短板，这通常需要结合转录组数据（RNA-seq）作为外部证据来修正剪接位点。

3.4.2. BLASTP 验证的必要性

实验结果显示，通过 BLASTP 比对，绝大多数（如 g2 匹配 YG13B_YEAST，相似度 94.9%）预测基因都在 UniProt 数据库中找到了同源蛋白。这一步骤至关重要：

去伪存真：从头预测软件（如 Augustus）可能会产生假阳性结果（预测了基因但实际不存在）。BLASTP 的高分匹配证明了这些预测序列在生物学上是真实存在的且高度保守。

功能注释：Augustus 只能提供基因的结构（位置），而通过 BLASTP 关联到已知蛋白，我们赋予了这些预测基因功能（如 YG13B 的功能），完成了从结构注释到功能注释的跨越。

3.5. 基因组注释之 DNA 元件预测

3.5.1. 预测结果的生物学意义分析

启动子区域的特异性富集：

从距离分布直方图可以看出，预测到的 TATA-box 元件在距离下游基因转录起始位点（TSS）0 bp 至 200 bp 的区间内出现了极显著的峰值。已有研究表明，酿酒酵母的 TATA-box 通常分布在 TSS 上游 -40 bp 至 -120 bp 区域，这与高等真核生物（通常在 -30 bp 左右）有所不同。本实验的预测峰值完美覆盖了这一特征区间，证明了预测结果并非随机噪音，而是真实的生物学信号。

得分与位置的非线性关系：

散点图显示，PWM 得分与距离 TSS 的远近之间没有显著的线性相关性（Pearson 相关系数接近 0）。这意味着，一个序列与 TATA-box 共有序列（Consensus Sequence）的相似度高低（得分高低），并不直接决定它在启动子上的精确位置。只要该元件位于有效的调控窗口内，且得分超过一定阈值，即可发挥转录起始功能。

元件分布的稀疏性：

得分分布直方图呈显著的右偏分布。绝大多数序列的得分集中在较低区间（13-14 分），而高得分（>16 分）的完美匹配序列非常稀少。这符合转录因子结合位点在基因组中“广泛存在低亲和力位点，稀缺高亲和力位点”的分布规律。

3.5.2. 模型适用性与跨物种保守性探讨

本实验的一个特殊之处在于：使用了植物的 TATA-box 频率矩阵来预测酿酒酵母的基因组。

保守性的验证：尽管跨越了界级别的物种差异，实验依然在正确的位置（TSS 上游）检测到了信号峰值。这强有力地证明了 TATA-box 核心序列（TATAWAW）在真核生物进化过程中的高度保守性。

局限性与改进：虽然预测成功，但在距离分布图中仍能观察到一定量的背景噪音（即非启动子区域的预测点）。这可能是因为植物和酵母在 TATA-box 侧翼序列的偏好性上存在差异。如果使用 EPD 数据库中酿酒酵母专用的 PWM 矩阵，理论上能进一步提高信噪比，降低假阳性率。

3.5.3. 阈值设定策略的讨论

实验中涉及两个关键阈值：PWM 得分阈值和 P-value 阈值。

当前策略：本实验采用了 Raw Score > Max_Score * 0.75 结合 P-value < 0.05 的筛选策略。这种双重标准既保证了序列的相似性，又通过随机打乱测试（Shuffle）排除了碱基组成偏差带来的干扰。

优化思路：观察得分直方图，如果后续实验需要极高的精确度（Precision），可以将 PWM 得分阈值提高至 15.5 分以上，这将大幅减少预测数量，但留下的每一个位点都将具有极高的置信度；反之，若用于探索性研究（提高 Recall），则可维持当前阈值。

3.6. 基因组数据可视化

3.6.1. 大数据可视化的核心机制：Range Request 技术的必要性

在实验过程中，初期遇到的 Error: HTTP 200 报错揭示了基因组可视化工具处理大数据的核心机制。

分片读取原理：基因组文件（如 GFF3.gz, BAM）通常体积巨大。JBrowse 不会下载整个文件，而是利用 HTTP 协议的 Range Requests 功能，仅请求当前屏幕可视窗口对应的字节片段（例如 bytes=1000-2000）。

索引的作用：实验中生成的 .tbi (Tabix) 和 .fai 索引文件正是为了支持这种随机读取。

实验启示：这一报错经历证明了在生物信息学 Web 开发中，服务器环境必须支持断点续传（Partial Content），这是区别于普通网页服务器的关键技术要求。

3.6.2. 可视化在多组学整合中的方法论意义

虽然单一的算法（如 Augustus 或 BLAST）能给出预测列表，但可视化是将结构化数据转化为生物学洞见的关键桥梁 。

打破“黑盒”：纯文本的 GFF3 文件无法直观展示基因结构的合理性。通过可视化，我们将一维的序列信息转化为二维的特征图谱，使得“多外显子”、“可变剪接”等抽象概念变得具象。

多维证据整合：本实验成功将从头预测（计算生物学）、同源比对（进化生物学）和元件预测（分子生物学）三个维度的数据叠加。这种多源证据互证

（Cross-validation）策略，能够有效弥补单一算法的缺陷（如 Augustus 对 UTR 区预测的不足），是现代基因组注释的金标准流程。